



数字逻辑电路



欢迎同学们来到奇妙的数字电路世界！

我们正处在高速发展的信息和智能时代，如何对各种各样的信息进行描述、存储、传输和处理呢？应该说，迄今为止人类找到的最佳信息表达形式是“数字”。

数字电路及其系统已经成为各行各业，乃至人们日常生活的重要组成部分，其无处不在。

秒懂百科

数字电路是指用数字信号



数字技术的发展是以电子器件的发展为基础的，经历了电子管、晶体管到集成电路的发展过程。

■ 数字技术的发展

➤ 布尔代数

乔治·布尔（George Boole, 1815~1864），1815年11月2日生于英格兰，19世纪最重要的数学家之一。1854年，他在出版的《思维规律的研究》中，提出了布尔代数。

布尔代数为数字电路设计奠定了数学基础。



➤ 电子管

李·德福雷斯特 (Lee de Forest , 1873 ~1961, 美国发明家), 1906年发明了真空三极管 (Vacuum Tube) , 也称电子管。

该发明是电子技术史上的一个里程碑, 它使得电子信号的处理和控制成为可能, 开启了电子时代。





➤ 数字电子学的诞生

克劳德·香农（ Claude Elwood Shannon ， 1916~2001， 美国数学家）， 1938年， 发表了论文“**继电器与开关电路的符号分析**”， 把布尔代数和继电器、电子电路开关结合起来， 诞生了数字电子学。

布尔代数
(1854年)

+

电子管
(1906年)

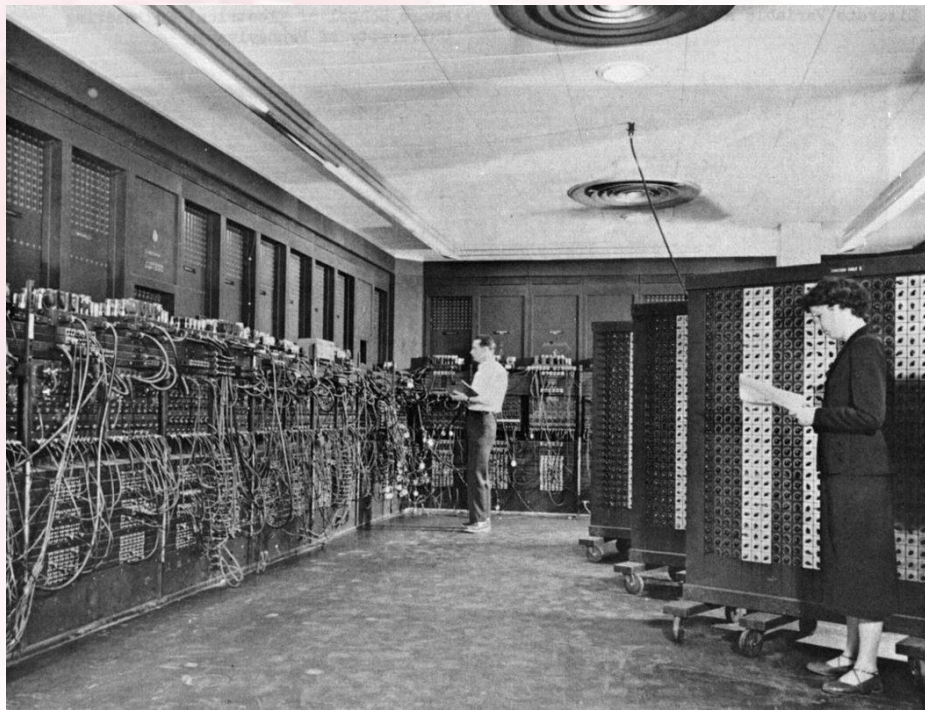


数字电子学
(1938年)



● 第一台电子管计算机

世界上第一台电子管计算机—ENIAC，1946年，有18800只电子管、重达30吨、占地面积170平方米、耗电约为140千瓦/小时，运算速度为5000次加法/秒。

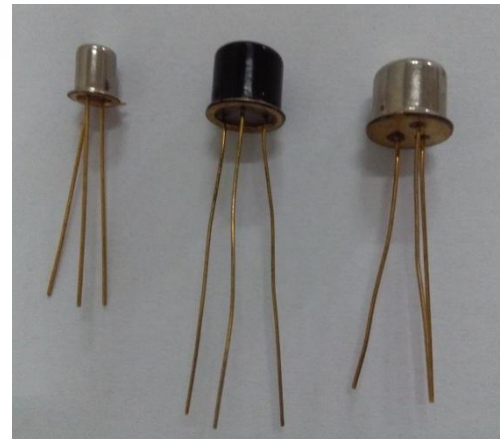




► 晶体管

William Shockley, Walter Houser Brattain和John Bardeen, 1947年发明了“点接晶体管放大器”，这就是后来引发一场电子革命的晶体管(Transistor)。

晶体管的问世，是20世纪的一项重大发明。人们就能用一个小巧的、消耗功率低的电子器件来代替体积大、功率消耗大的电子管。





➤ 集成电路

杰克·基尔比 (Jack Kilby, 1923~2005, 美国电气工程师), 1958年研制出了集成电路(Integrated Circuit), 只集成5个元件。

与分立元件电路相比, 集成电路具有体积小、重量轻、可靠性高等优点。

- ▲ 小规模集成电路(SSI), 10~100 个元件/片。
- ▲ 中规模集成电路(MSI), 100~1,000个元件/片。
- ▲ 大规模集成电路(LSI), 1,000~1万个元件/片。
- ▲ 超大规模集成电路(VLSI), 1万~100万个元件/片。
- ▲ 甚大规模集成电路 (ULSI), 100万~ 1000万元件/片。
- ▲ 极大规模集成电路 (GSI), 1000万~元件/片。





■ 课程的性质与作用：

“数字逻辑电路”课程是电子信息类、电气类、自动化类、计算机类等专业的专业基础课。

提供数字技术和数字系统的基本概念、基本原理和基本技能，培养工程实践能力，为后续专业课程的学习以及适应现代信息社会的快速变化奠定坚实的基础。

■ 课程的目标：

理解数字技术的基本概念、基本原理；掌握数字电路的分析与设计方法；学会使用EDA工具设计数字电路；能够对数字电路进行测试并根据测试结果分析、判断和排除故障；具有设计和实现复杂数字系统的初步能力。



课程 内 容

第1章 数字逻辑基础

第2章 逻辑门电路

第3章 组合逻辑电路

第4章 常用组合逻辑功能器件

第5章 时序逻辑电路

第6章 常用时序逻辑功能器件

第7章 半导体存储器和可编程逻辑器件

第8章 脉冲信号的产生与整形

第9章 数模和模数转换



参考教材

1. 蒋立平主编：数字逻辑电路与系统设计（第4版），电子工业出版社。
2. 阎石主编：数字电子技术基础（第六版），高等教育出版社。
3. Thomas L.Floyd著，Digital Fundamentals (Eleventh Edition), 科学出版社（英文影印版）。

“爱课程” 中国大学MOOC, SPOC课程：数字逻辑电路

<https://www.icourse163.org/spoc/course/NJUST-1449972193>



绪 论

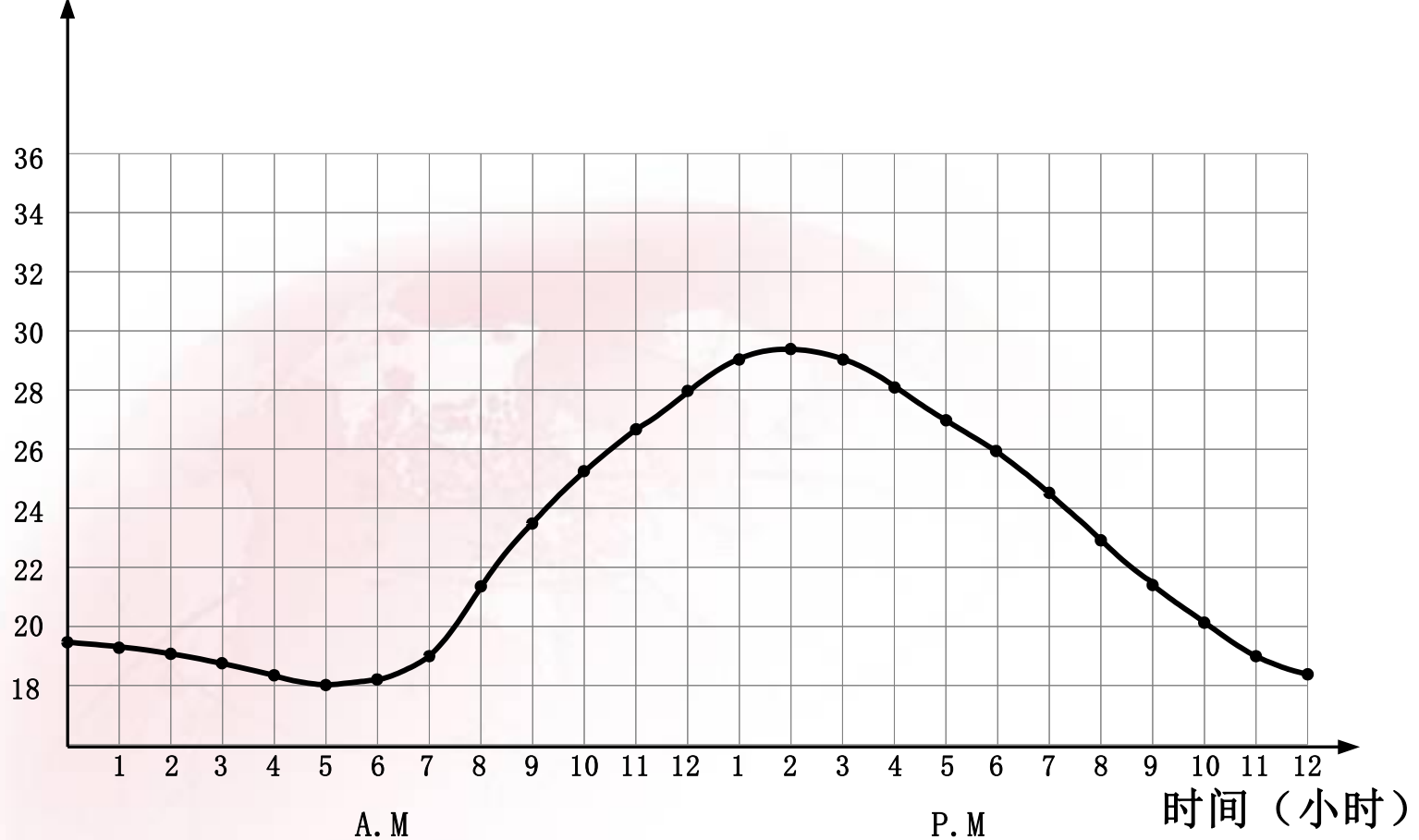
一、模拟量和数字量

模拟量：模拟量就是在时间和幅值上均**连续变化**的量。自然界中可测试的物理量一般都是模拟量，例如温度，压力，距离，时间等。

数字量：数字量是在时间和幅值上均**离散**的量。数字量一般是将模拟量经过**抽样**、**量化**和**编码**后而得到的。



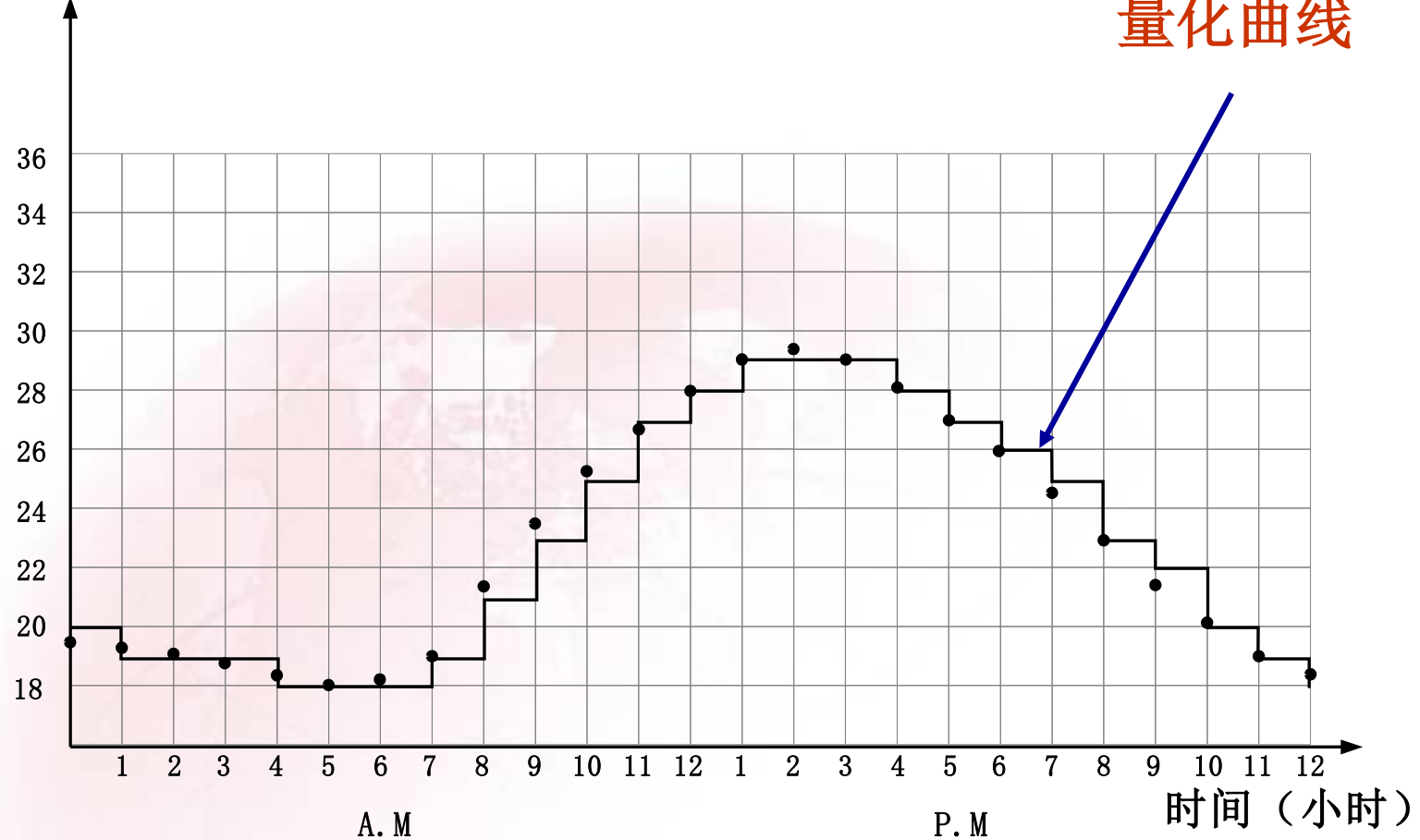
温度 ($^{\circ}\text{C}$)



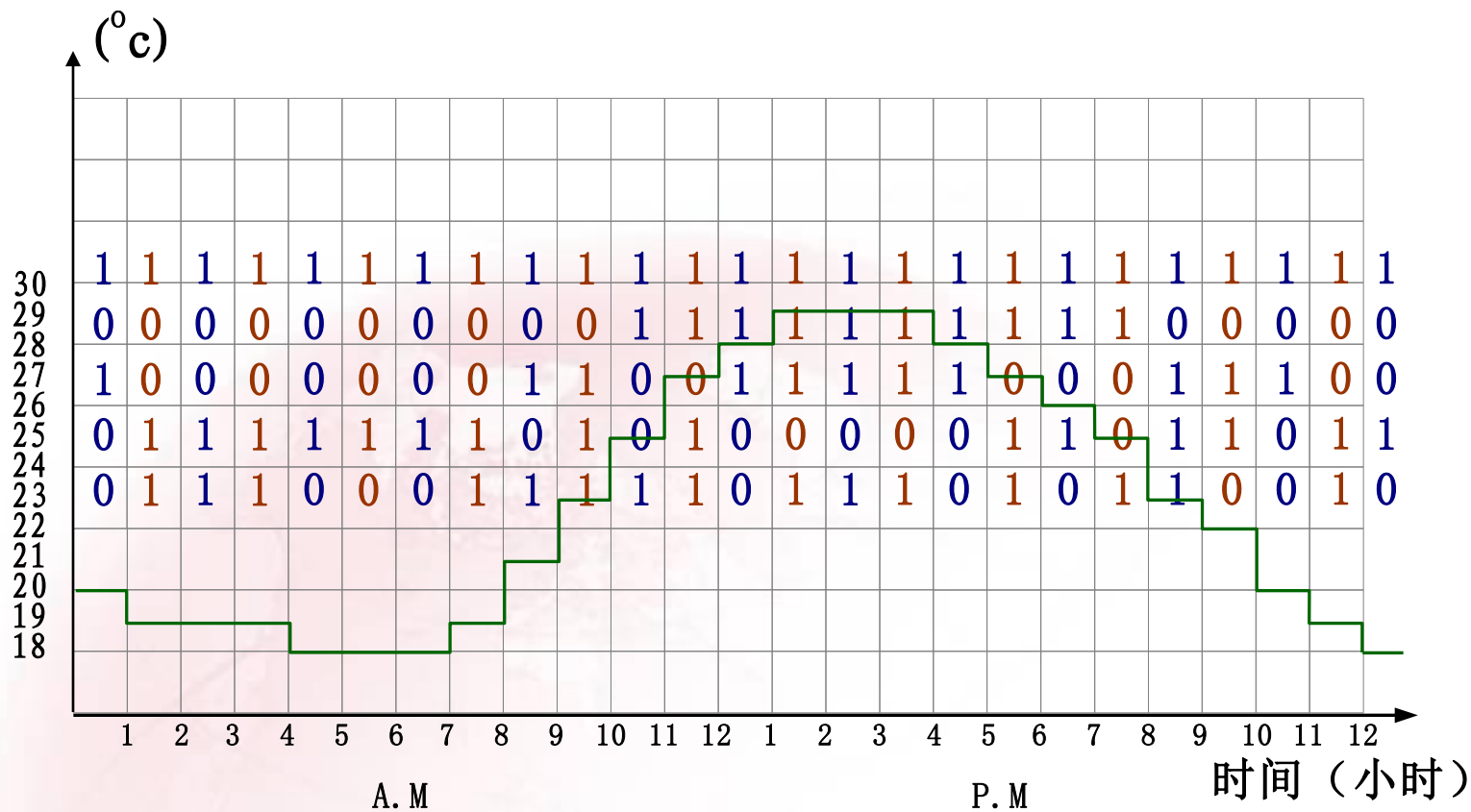
温度和时间关系图(用模拟量表示)



温度 ($^{\circ}\text{C}$)



温度和时间关系图(用采样值表示)



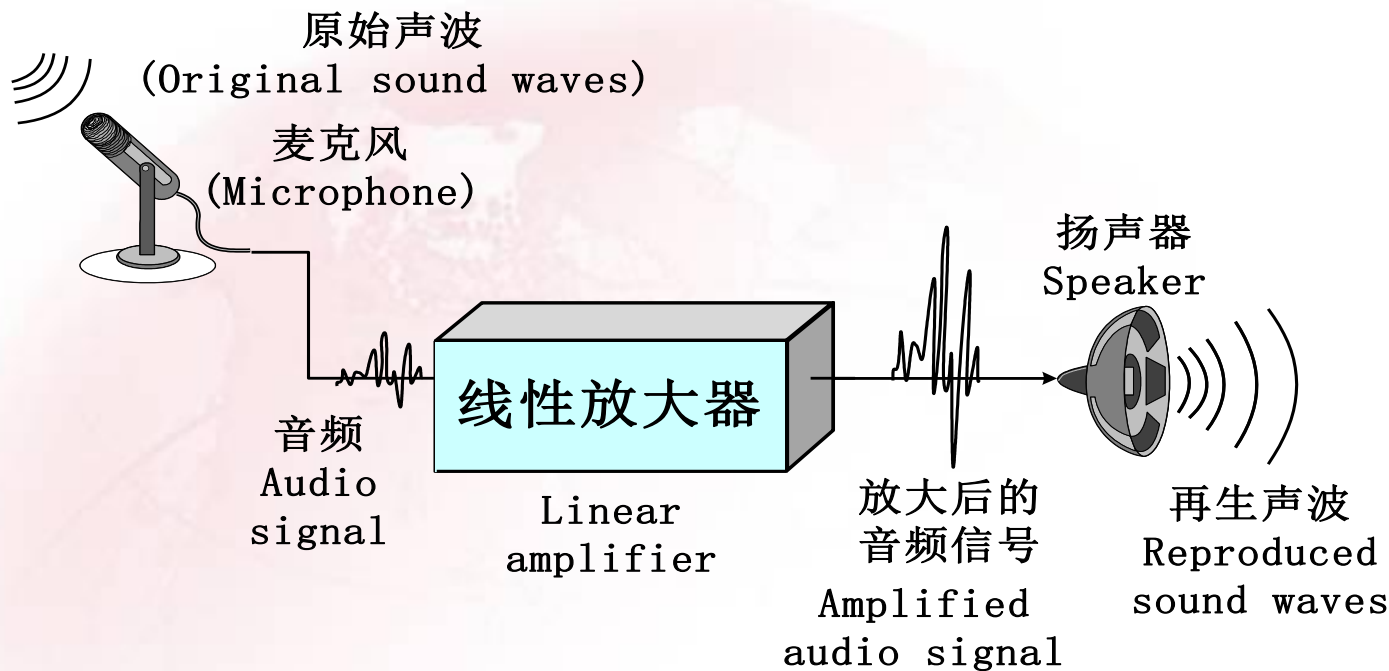
温度和时间关系图(用数字形式表示)



二、模拟和数字系统的几个实例

1. 音频有线扩音系统

音频有线扩音系统为纯模拟系统。

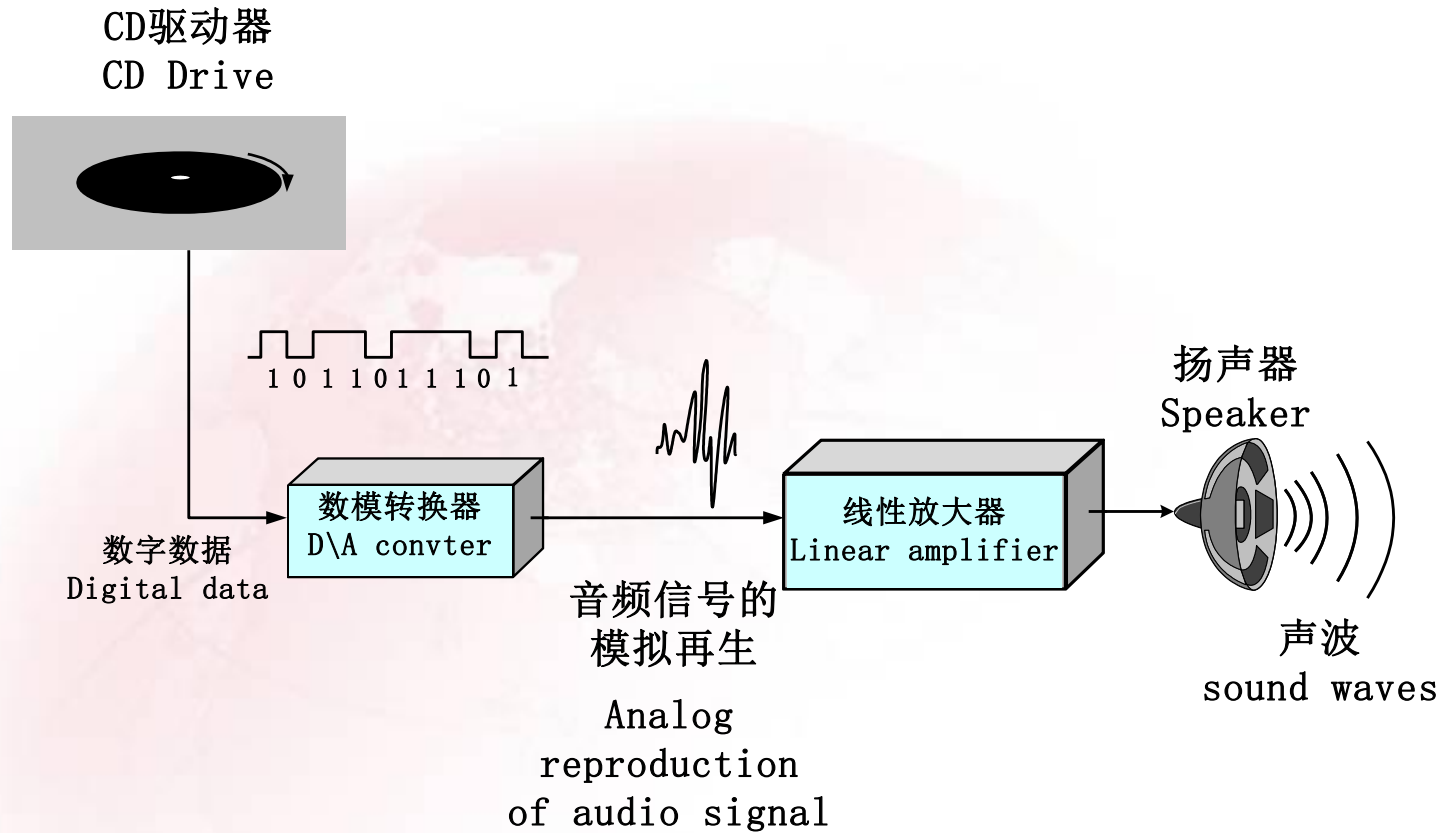


音频有线扩音系统
Audio public address system



2. CD 播放机

CD 播放机为数模混合系统。

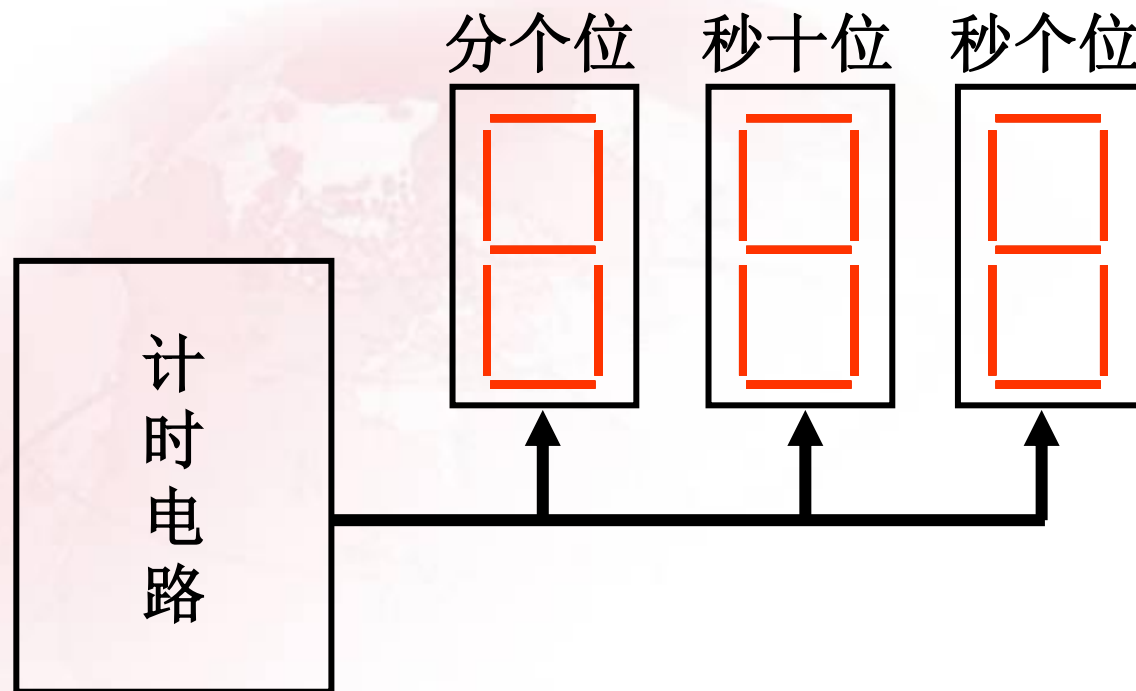


CD机原理图（单声道）
Basic principle of a CD player



3. 数字钟

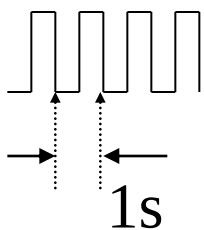
带**数字显示**的数字钟是一个纯数字系统，以三位计时系统为例。



三位计时器示意图

定时激励信号产生电路

秒脉冲



脉冲个数记录电路

分个
位二
进制
码

秒十
位二
进制
码

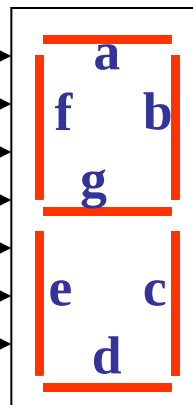
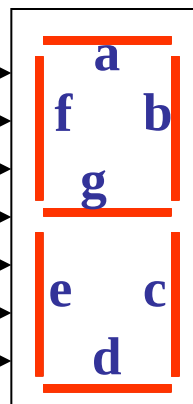
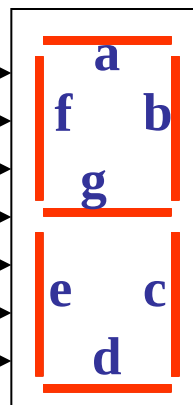
秒个
位二
进制
码

码转换电路（译码器）

分个
位显
示码

秒十
位显
示码

秒个
位显
示码





三、数字电路特点

1. 工作信号是二进制的数字信号，即只有“0”和“1”两种取值；
2. 数字电路中，可以很方便地用电子器件的开关来实现二值数字逻辑；
3. 数字电路既能进行数值运算，也能进行逻辑判断和逻辑运算；
4. 电路工作可靠，精度高，抗干扰性能好；
5. 数字信号便于保存、传输、保密性好。



第1章 数字逻辑基础

1.1 数制与数制转换

所谓“数制”，指进位计数制，即用进位的方法来计数。

数制包括计数符号（数码）和进位规则两个方面。

常用数制有二进制、十进制、八进制、十六进制、六十进制等。



1.1.1 常用数制

1. 十进制

(1) 计数符号: **0, 1, 2, 3, 4, 5, 6, 7, 8, 9**

(2) 进位规则: **逢十进一**

(3) 十进制数按权展开式

例: $1987.45 = 1 \times 10^3 + 9 \times 10^2 + 8 \times 10^1 + 7 \times 10^0$
 $+ 4 \times 10^{-1} + 5 \times 10^{-2}$



$$(N)_{10} = \sum_{i=-m}^{n-1} a_i \times 10^i$$

系数 $\xrightarrow{\quad}$ $\xleftarrow{\quad}$ 权

2. 二进制

(1) 计数符号: **0, 1**

(2) 进位规则: **逢二进一**

(3) 二进制数按权展开式 $(N)_2 = \sum_{i=-m}^{n-1} a_i \times 2^i$

$$(101.11)_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}$$



数字电路中采用二进制的原因：

- ▲ 数字装置简单可靠；
- ▲ 二进制数运算规则简单；
- ▲ 数字电路既可以进行数值运算，也可以进行逻辑运算。

3.十六进制和八进制

十六进制数计数符号：0,1,.,9,A,B,C,D,E,F

十六进制数进位规则：逢十六进一

按权展开式：
$$(N)_{16} = \sum_{i=-m}^{n-1} a_i \times 16^i$$



例:

$$\begin{aligned}(6D.4B)_{16} &= 6 \times 16^1 + D \times 16^0 + 4 \times 16^{-1} + B \times 16^{-2} \\ &= 6 \times 16^1 + 13 \times 16^0 + 4 \times 16^{-1} + 11 \times 16^{-2}\end{aligned}$$

八进制数计数符号: **0,1,...,6,7**

八进制数进位规则: **逢八进一**

按权展开式: $(N)_8 = \sum_{i=-m}^{n-1} a_i \times 8^i$

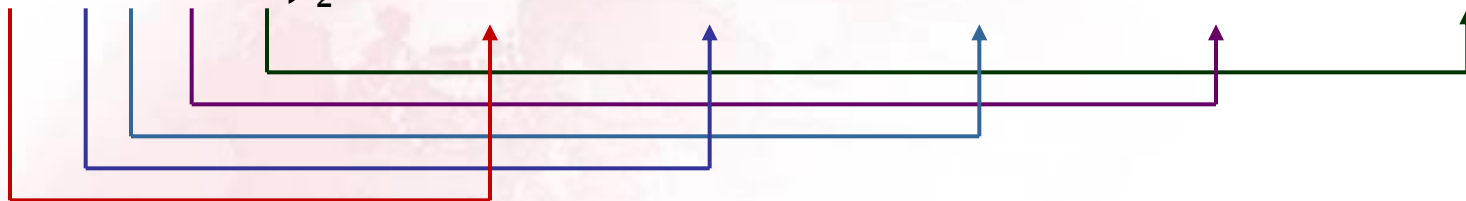
例: $(63.45)_8 = 6 \times 8^1 + 3 \times 8^0 + 4 \times 8^{-1} + 5 \times 8^{-2}$



4. 二进制数与十进制数之间的转换

(1) 二进制数转换为十进制数（按权展开法）

例： $(1011.101)_2 = 1 \times 2^3 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-3}$



$$= 8 + 2 + 1 + 0.5 + 0.125$$

$$= (11.625)_{10}$$

▲ 还可以采用**基数连乘、连除**等方法。



(2) 十进制数转换为二进制数（提取2的幂法）

例： $(45.5)_{10} = 32 + 8 + 4 + 1 + 0.5$

$$= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1}$$
$$= (101101.1)_2$$

▲ 还可以采用**基数连除、连乘**等方法。



1.2 几种简单的编码

1. 二 - 十进制码 (**BCD码**)(**Binary Coded Decimal codes**)

用四位二进制代码来表示一位十进制数码，这样的代码称为二-十进制码，或BCD码。

四位二进制有**16**种不同的组合，可以在这**16**种代码中任选**10**种表示十进制数的10个不同符号，选择方法很多。选择方法不同，就能得到不同的编码形式。

常见的BCD码有8421码、5421码、2421码、余3码等。



常用BCD码

十进制数	8421码	5421码	2421码	余3码
0	0000	0000	0000	0011
1	0001	0001	0001	0100
2	0010	0010	0010	0101
3	0011	0011	0011	0110
4	0100	0100	0100	0111
5	0101	1000	1011	1000
6	0110	1001	1100	1001
7	0111	1010	1101	1010
8	1000	1011	1110	1011
9	1001	1100	1111	1100



(1) **有权BCD码**：每位数码都有确定的位权的码。

例如：8421码、5421码、2421码

如：5421码1011代表 $5+0+2+1=8$

2421码1100代表 $2+4+0+0=6$

▲ **5421BCD码和2421BCD码不唯一。**

例：2421BCD码**0110**也可表示**6**，今后一律按表中规律编码。



☆ 在表中：

- ① **8421BCD**码和代表0~9的二进制数一一对应；
- ② **5421BCD**码的前5个码和**8421BCD**码相同，后5个码在前5个码的基础上加1000构成，这样的码，前5个码和后5个码一一对应相同，仅高位不同；
- ③ **2421BCD**码的前5个码和**8421BCD**码相同，后5个码以**中心对称取反**，这样的码称为**自反代码**。

例： 4→0100 5→1011
 0→0000 9→1111



(2) **无权BCD码**：每位数码无确定的位权，如：余3码。

余3码的编码规律为：在8421BCD码上加0011。

例：6的余3码为： $0110 + 0011 = 1001$

2. 格雷码(**Gray**码)

格雷码为无权码，特点为：相邻两个代码之间仅有一位不同，其余各位均相同。具有这种特点的代码称为**循环码**，格雷码是**循环码**。

这种编码方式由弗兰克·格雷（Frank Gray）在1947年提出，最初用于减少数字通信中出错的可能性。



格雷码和四位二进制码之间的关系：

设四位二进制码为 $B_3B_2B_1B_0$ ，格雷码为 $R_3R_2R_1R_0$ ，

则：

$R_3=B_3,$	其中， $\oplus$ 为 异或 运算符，其运算规则为：若两运算数 相同 ，结果为“0”；两运算数 不同 ，结果为“1”。
$R_2=B_3 \oplus B_2$	
$R_1=B_2 \oplus B_1$	
$R_0=B_1 \oplus B_0$	

对于n位： $R_n=B_n$ $R_i=B_{i+1} \oplus B_i$

同时有： $B_3=R_3,$

$$B_2=B_3 \oplus R_2$$

$$B_1=B_2 \oplus R_1$$

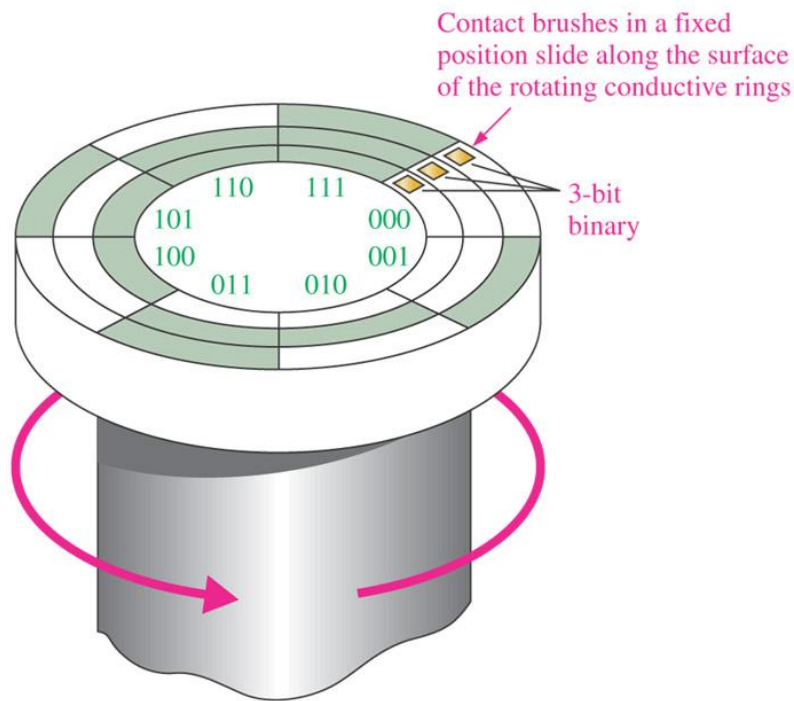
$$B_0=B_1 \oplus R_0$$

对于n位： $B_n=R_n$ $B_i=B_{i+1} \oplus R_i$

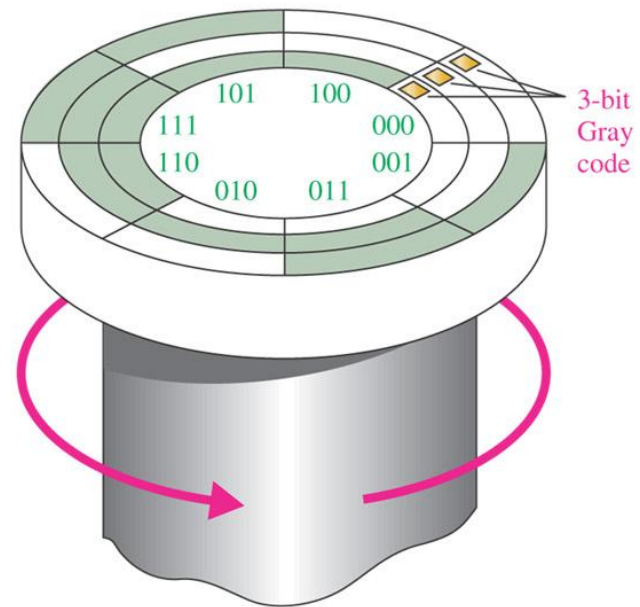


格雷码应用示例

一个3位轴角位置编译器机械构造的简化图如下所示，有三个同心传导圆圈被分成八个扇区。



(a) Binary



(b) Gray code



3. 奇偶校验码

奇偶校验码是一种具有检错能力的代码，它是在原代码的基础上增加一个码位使代码中含有的1的个数均为奇数（称为奇校验）或偶数（称为偶校验），通过检查代码中含有的1的奇偶性来判别代码的合法性。

4. 字符数字码

字符数字码能表示计算机键盘上能看到的各种符号和功能键。ASCII码是目前国际上最通用的一种字符数字码，它是用7位二进制码来表示128个十进制数、英文大小写字母、控制符、运算符以及特殊符号。



■ 数制与数制转换、数制与编码转换

例：用8421BCD码表示十进制数 $(73.5)_{10}$

十进制数	7	3	.	5
	↓	↓		↓
8421BCD码	0111	0011	.	0101

故： $(73.5)_{10} = (01110011.0101)_{8421\text{BCD码}}$

思考： $(73.5)_{10} = (1001001.1)_2$

$(00010101.0101)_{8421\text{BCD码}} = (1111.1)_2$

$(10110.1)_2 = (00100010.0101)_{8421\text{BCD码}}$

$(1001)_{5421\text{BCD}} + (1001)_{\text{余3码}} = (00010010)_{8421\text{BCD}}$

$(100110)_2 = (110101)_{\text{格雷码}}$

$(86)_{10} = (126)_8$



1.3 算术运算

1.3.1 无符号二进制数的算术运算

1. 加法

$$0+0=0$$

$$1+0=0+1=1$$

$$1+1=\boxed{1}0$$

向高一位的进位

$$\begin{array}{r} 1011 (11) \\ + 0110 (6) \\ \hline 10001 (17) \end{array}$$

2. 减法

$$0-0=0$$

$$1-1=0$$

$$1-0=1$$

$$0-1=\boxed{-1}1$$

向高一位的借位

$$\begin{array}{r} 1011 (11) \\ - 0110 (6) \\ \hline 0101 (5) \end{array}$$



3. 乘法

$$\begin{aligned}0 \times 0 &= 0 \\1 \times 0 &= 0 \times 1 = 0 \\1 \times 1 &= 1\end{aligned}$$

$$\begin{array}{r}1101 \quad (13) \\ \times 0101 \quad (5) \\ \hline 1101 \\ 0000 \\ 1101 \\ + 0000 \\ \hline 1000001 \quad (65)\end{array}$$

4. 除法

$$\begin{aligned}0 \div 1 &= 0 \\1 \div 1 &= 1\end{aligned}$$



1.3.2 有符号二进制数的表示

带符号二进制数中最左边的一位是符号位，用来指明这个数是正数还是负数。**符号位为0表示正数，为1表示负数。**

在二进制中，带符号整数有三种表达方式：**原码、反码和补码。**

对于**正数**：原码、反码和补码均是相同的。

对于**负数**：

在**原码**形式中，负数和其相应的正数具有相同的数值位，但符号位为1而不是0。

在**反码**形式中，负数是将其相应正数**按位取反后**而得到的。

在**补码**形式中，负数是将其相应正数**按位取反后加1**而得到的。

说明： 0的原码有两种形式： $+0=(000 \cdots 0)_{\text{原码}}$ ， $-0=(100 \cdots 0)_{\text{原码}}$

0的反码有两种形式： $+0=(000 \cdots 0)_{\text{反码}}$ ， $-0=(111 \cdots 1)_{\text{反码}}$

0的补码表示是唯一的，即 $(000 \cdots 0)_{\text{补码}}$



例如：以原码、反码和补码的形式将十进制数-39表示为一个8位二进制数。

解：首先，十进制数+39的8位二进制数可以表示为：**00100111**。

在**原码**形式中，-39是通过**改变符号位为1**并保持数值位不变而得到的，即-39的原码形式为：**10100111**。

在**反码**形式中，-39是通过**将+39（00100111）按位取反**后而得到的，即-39的反码形式为：**11011000**。

在**补码**形式中，-39是通过**将+39（00100111）按位取反后加1**而得到的，即-39的补码形式为：**11011001**。



1.3.3 二进制补码的加法运算

第一种情况：两个数都是正数（以8位带符号数为例）

$$\begin{array}{r} 00000111 \\ + 00000100 \\ \hline 00001011 \end{array} \qquad \begin{array}{r} 7 \\ + 4 \\ \hline 11 \end{array}$$

第二种情况：正数的数值大于负数的数值（以8位带符号数为例）

$$\begin{array}{r} 00001111 \\ + 11111010 \\ \hline 1\ 00001001 \end{array} \qquad \begin{array}{r} 15 \\ + -6 \\ \hline 9 \end{array}$$

丢弃进位 $\longrightarrow$



第三种情况：负数的数值大于正数的数值（以8位带符号数为例）

$$\begin{array}{r} 00010000 \\ + 11101000 \\ \hline 11111000 \end{array} \qquad \begin{array}{r} 16 \\ + -24 \\ \hline -8 \end{array}$$

第四种情况：两个数都是负数（以8位带符号数为例）

$$\begin{array}{r} 11111011 \\ + 11110111 \\ \hline 111110010 \end{array} \qquad \begin{array}{r} -5 \\ + -9 \\ \hline -14 \end{array}$$

丢弃进位 →

$$\underline{1} \ 11110010$$



■ 溢出条件

当两个数相加时，如果表示和所需要的位数超出了这两个数的位数，就会发生溢出，溢出的结果是出现了错误的符号位。

溢出仅发生在两个都是正数或者两个都是负数的情况下，如果相加结果的符号位与两个相加数的符号位不同，则表明产生了溢出。

以8位带符号数为例说明：

01111101	125
+ 00111010	+ 58
符号不正确 → 10110111	183
大小不正确 →	

结果183，需要8个数值位，由于数中只有7个数值位（有一位是符号位），这时进位就会进入符号位，从而产生溢出。



1.4 逻辑代数中的逻辑运算

研究数字电路的基础为**逻辑代数**，由英国数学家**George Boole**提出的，逻辑代数也称**布尔代数**。

在逻辑代数中，变量常用字母 **A,B,C,.....Y,Z,a,b,c,.....x,y,z** 等表示，变量的取值只能是“**0**”或“**1**”。

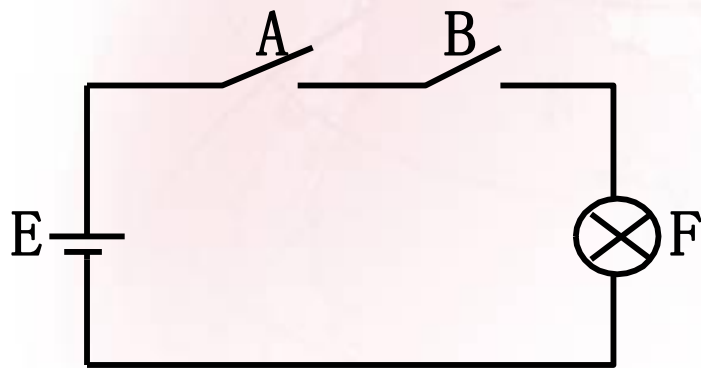
逻辑代数中只有三种基本逻辑运算，即“**与**”、“**或**”、“**非**”，以及在基本逻辑运算基础上构成的复合逻辑运算。



1.4.1 基本逻辑运算

1. 与逻辑运算

定义：只有决定一事件的**全部**条件都具备时，这件事才成立；如果有一个或一个以上条件不具备，则这件事就不成立。这样的因果关系称为“**与**”逻辑关系。



与逻辑电路

与逻辑电路状态表

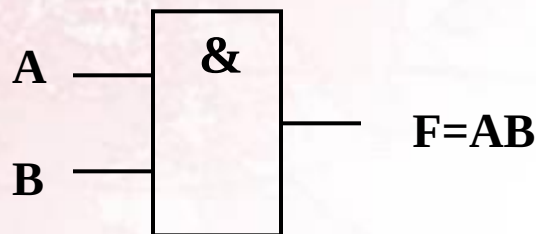
开关A状态	开关 B状态	灯F状态
断	断	灭
断	合	灭
合	断	灭
合	合	亮



若将开关断开和灯的熄灭状态用逻辑量“0”表示；将开关合上和灯亮的状态用逻辑量“1”表示，则上述状态表可表示为：

与逻辑真值表

A	B	$F=A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1



与门逻辑符号

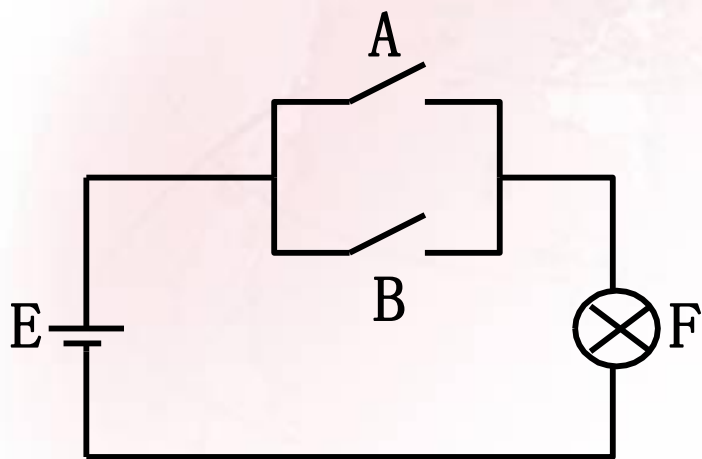
与门的逻辑功能概括：
“有0出0，全1出1”

真值表：把所有输入变量取值的各种可能组合和对应的输出变量值之间的逻辑关系列成表格的形式。



2. 或逻辑运算

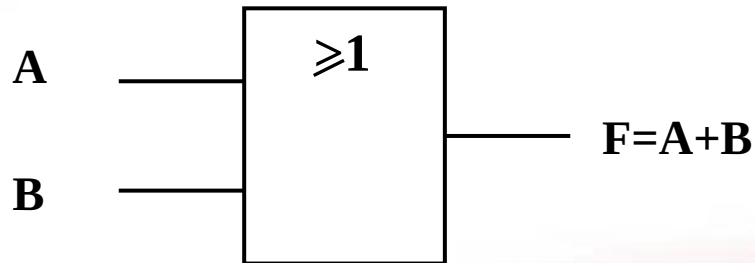
定义：在决定一事件的各种条件中，只要有一个或一个以上条件具备时，这件事就成立；只有所有的条件都不具备时，这件事就不成立。这样的因果关系称为“或”逻辑关系。



或逻辑电路

或逻辑真值表

A	B	$F = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

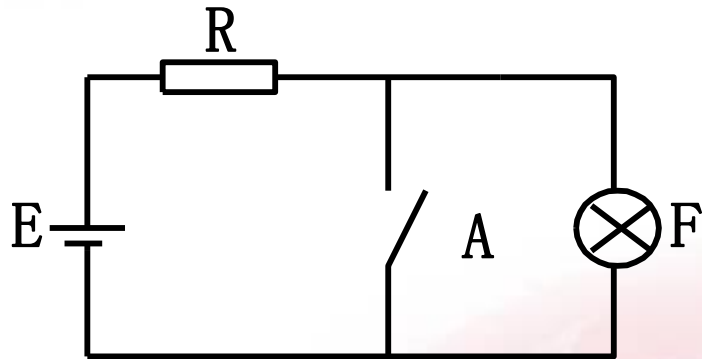


或门逻辑符号

或门的逻辑功能概括为：
“有1出1，全0出0”

3. 非逻辑运算

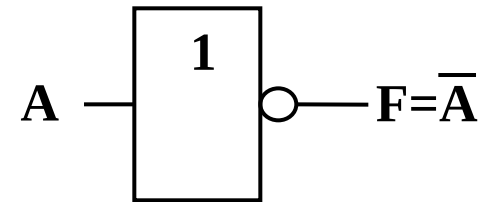
定义：假定事件F成立与否同条件A的具备与否有关，若A具备，则F不成立；若A不具备，则F成立。F和A之间的这种因果关系称为“非”逻辑关系。



非逻辑电路

非逻辑真值表

A	$F = \bar{A}$
0	1
1	0



非门逻辑符号

- 与门和或门均可以有多个输入端，一个输出端。
- 非门只有一个输入端，一个输出端。

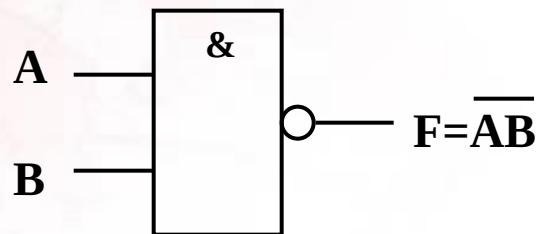


1.4.2 复合逻辑运算

1. 与非逻辑 (将与逻辑和非逻辑组合而成)

与非逻辑真值表

A	B	$F = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0



与非门逻辑符号

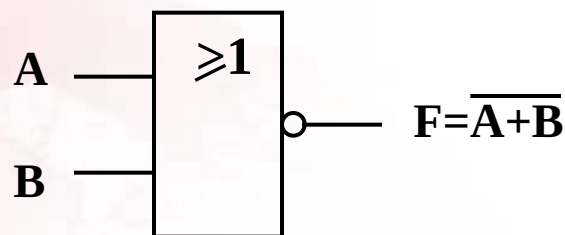
与非门的逻辑功能概括为：“有0出1，全1出0”



2. 或非逻辑 (将或逻辑和非逻辑组合而成)

或非逻辑真值表

A	B	$F = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0



或非门逻辑符号

或非门的逻辑功能概括为：“全0出1，有1出0”

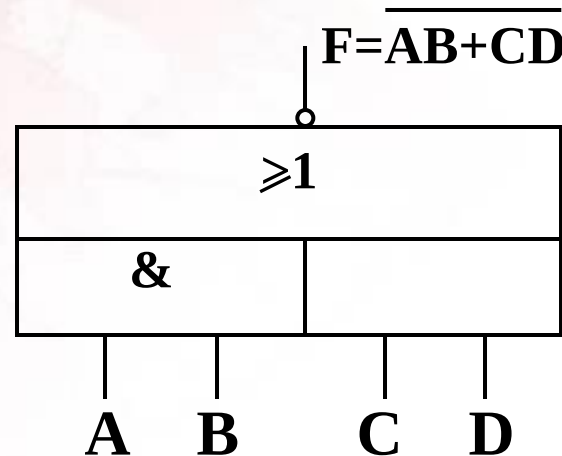


3. 与或非逻辑 (由与、或、非三种逻辑组合而成)

与或非逻辑函数式:

$$F = \overline{AB + CD}$$

与或非门的逻辑符号



与或非门的逻辑功能概括为:“每组有0出1, 某组全1出0”

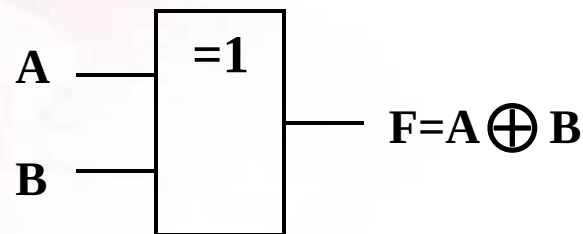


4. 异或逻辑

异或逻辑的函数式为： $F = \overline{A}B + A\overline{B} = A \oplus B$

异或逻辑真值表

A	B	$F = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

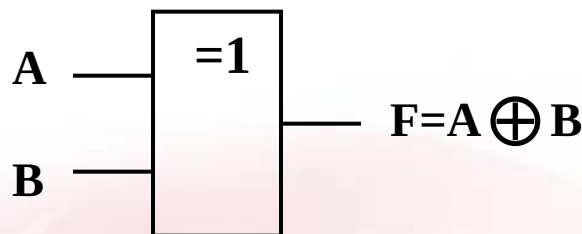


异或门逻辑符号

异或逻辑的功能为：“相同出0，相异出1”



应用：



异或门逻辑符号

若B作为控制端，A作为信号输入端：

B=0时， $F=A$ ；

B=1时， $F=\bar{A}$

在大规模集成电路中，可作为极性控制电路使用。

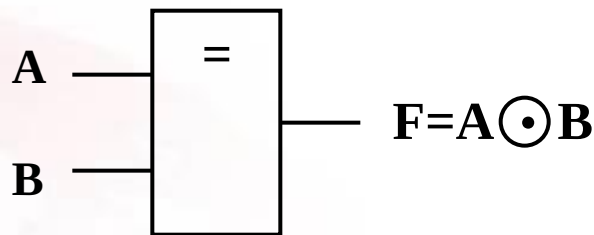


5. 同或逻辑

同或逻辑式为： $F = \overline{A} \overline{B} + A B = A \odot B$

同或逻辑真值表

A	B	$F = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1



同或门逻辑符号

同或逻辑的功能为：“相同出1，相异出0”

对照同或和异或逻辑真值表，可以发现：两个变量的同或和异或互为反函数，即： $A \oplus B = \overline{A \odot B}$ ， $A \odot B = \overline{A \oplus B}$ 。

■ 本课程中，门电路的表示方法均采用“国标”符号。



1.4.3 正逻辑与负逻辑

1. 逻辑状态和逻辑电平

(1) 逻辑状态: { 逻辑1状态
逻辑0状态

(2) 逻辑电平: { 逻辑高电平, 以 V_H 表示
逻辑低电平, 以 V_L 表示



2. 正、负逻辑

门电路的输入、输出为二值信号，用“0”和“1”表示。这里的“0”、“1”一般用两个不同电平值来表示。

(1) 若用高电平 V_H 表示逻辑“1”，用低电平 V_L 表示逻辑“0”，则称为正逻辑约定，简称正逻辑；

(2) 若用高电平 V_H 表示逻辑“0”，用低电平 V_L 表示逻辑“1”，则称为负逻辑约定，简称负逻辑。

在本课程中，如不作特殊说明，一般都采用正逻辑表示。



V_H 和 V_L 的具体值，由所使用的集成电路品种以及所加电源电压而定。

例如：TTL集成电路，电源电压为5V，

高电平 V_H 范围为2V~5V，

低电平 V_L 范围为0V~0.8V。



3. 对一个特定的逻辑门，采用不同的逻辑表示时，其门的名称也就不同。

正负逻辑转换举例

电平真值表			正逻辑(与非门)			负逻辑(或非门)		
V_{i1}	V_{i2}	V_o	A	B	Y	A	B	Y
V_L	V_L	V_H	0	0	1	1	1	0
V_L	V_H	V_H	0	1	1	1	0	0
V_H	V_L	V_H	1	0	1	0	1	0
V_H	V_H	V_L	1	1	0	0	0	1



1.5 逻辑代数的基本定律和规则

1.5.1 逻辑函数的相等

设有两个逻辑： $F_1=f_1(A_1,A_2,\dots,A_n)$

$$F_2=f_2(A_1,A_2,\dots,A_n)$$

如果对于 $A_1,A_2,\dots,A_n$ 的任何一组取值(共 2^n 组)， F_1 和 F_2 均相等, 则称 F_1 和 F_2 相等。

因此，如两个函数的真值表相等，则这两个函数一定相等。



例：设两个函数： $F_1=A+BC$

$$F_2=(A+B)(A+C)$$

求证： $F_1=F_2$

解：这两个函数都具有三个变量，有8组逻辑取值，可以列出 F_1 和 F_2 的真值表。

由表可见，对于A、B、C的每组取值，函数 F_1 的值和 F_2 的值均相等，所以 $F_1=F_2$ 。

A	B	C	F_1	F_2
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1



1.5.2 基本定律

- | | | | |
|--------|--------------------------------------|---|-------------------------------------|
| ① 0-1律 | $A \cdot 0 = 0$ | ; | $A + 1 = 1$ |
| ② 自等律 | $A \cdot 1 = A$ | ; | $A + 0 = A$ |
| ③ 重迭律 | $A \cdot A = A$ | ; | $A + A = A$ |
| ④ 互补律 | $A \cdot \bar{A} = 0$ | ; | $A + \bar{A} = 1$ |
| ⑤ 交换律 | $A \cdot B = B \cdot A$ | ; | $A + B = B + A$ |
| ⑥ 结合律 | $A(BC) = (AB)C$ | ; | $A + (B + C) = (A + B) + C$ |
| ⑦ 分配律 | $A(B + C) = AB + AC$ | ; | $A + BC = (A + B)(A + C)$ |
| ⑧ 反演律 | $\overline{A + B} = \bar{A} \bar{B}$ | ; | $\overline{AB} = \bar{A} + \bar{B}$ |
| ⑨ 还原律 | $\overline{\bar{A}} = A$ | | |

反演律也称**德·摩根**定理, 是一个非常有用的定理。



1.5.3 逻辑代数的三条规则

1. 代入规则

任何一个含有变量 X 的等式，如果将所有出现 X 的位置，都用一个逻辑函数式 F 代替，则等式仍然成立。

例：已知等式 $\overline{A+B} = \overline{A} \cdot \overline{B}$ ，有函数式 $F=B+C$ ，则

用 F 代替等式中的 B ：

$$\text{有 } \overline{A+(B+C)} = \overline{A} \cdot \overline{B+C}$$

$$\text{即 } \overline{A+B+C} = \overline{A} \cdot \overline{B} \cdot \overline{C}$$

由此可以证明反演定律对 n 变量仍然成立。

$$\overline{A_1+A_2+\dots+A_n} = \overline{A_1} \cdot \overline{A_2} \cdot \dots \cdot \overline{A_n}$$



2. 反演规则

设F为任意逻辑表达式，若将F中所有运算符、常量及变量作如下变换：

·	+	0	1	原变量	反变量
↓	↓	↓	↓	↓	↓
+	·	1	0	反变量	原变量

则所得新的逻辑式即为F的反函数，记为 $\bar{F}$ 。

例：已知 $F = \bar{A}B + A\bar{B}$ ，根据上述规则可得：

$$\bar{F} = (A + \bar{B})(\bar{A} + B)$$



例 已知 $F=A+B+\overline{C}+\overline{D}+\overline{\overline{E}}$, 则

$$\overline{F}=\overline{A} \overline{\overline{B}} \overline{C} \overline{\overline{D}} \overline{\overline{E}}$$

由F求反函数注意:

- (1) 保持原式运算的优先次序;
- (2) 原式中的不属于单变量上的非号不变。



3. 对偶规则

设F为任意逻辑表达式, 若将F中所有运算符和常量作如下变换:

•	+	0	1
↓	↓	↓	↓
+	•	1	0

则所得新的逻辑表达式即为F的对偶式, 记为**F'**。

例 有 $F = A \bar{B} + C \bar{D}$ $F' = (A + \bar{B})(C + \bar{D})$

例 有 $F = A + \overline{B + C + D + E}$ $F' = A \bar{B} \bar{C} \bar{D} \bar{E}$



对偶是相互的， F 和 F' 互为对偶式。求对偶式注意：

- ① 保持原式运算的优先次序；
- ② 原式中的长短“非”号不变；
- ③ 单变量的对偶式为自己。

■ 若有两个逻辑表达式 F 和 G 相等，则各自的对偶式 F' 和 G' 也相等。

使用对偶规则可使得一些表达式的证明更加方便。

例：已知 $A(B+C)=AB+AC \xrightarrow{\text{对偶关系}} A+BC=(A+B)(A+C)$



1.5.4 逻辑代数的常用公式

(1) 消去律 $AB + A\bar{B} = A$

证明：

$$AB + A\bar{B} = A(B + \bar{B}) = A \cdot 1 = A \xrightarrow{\text{对偶关系}} (A+B)(A+\bar{B}) = A$$

该公式说明：两个乘积项相加时，若它们**只有一个因子不同**（如一项中有**B**，另一项中有 **$\bar{B}$** ），而其余因子完全相同，则这两项可以合并成一项，且能消去那个不同的因子（即**B**和 **$\bar{B}$** ）。



(2) 吸收律1 $A+AB=A$

证明:

$$A+AB=A(1+B)=A\cdot 1=A \xrightarrow{\text{对偶关系}} A(A+B)=A$$

该公式说明：两个乘积项相加时，若其中一项是另一项的因子，则另一项是多余的。



(3) 吸收律2 $A + \bar{A}B = A + B$

证明：

$$A + \bar{A}B = (A + \bar{A})(A + B) = 1 \cdot (A + B) \xrightarrow{\text{对偶关系}} A(\bar{A} + B) = AB \\ = A + B$$

该公式说明：两乘积项相加时，若其中一项的非是另一项的因子，则此因子是多余的。



(4) 包含律

$$AB + \bar{A}C + BC = AB + \bar{A}C$$

证明：

$$AB + \bar{A}C + BC$$

$$= AB + \bar{A}C + (A + \bar{A})BC$$

$$= AB + \bar{A}C + ABC + \bar{A}BC$$

$$= AB(1 + C) + \bar{A}C(1 + B)$$

$$= AB + \bar{A}C$$

对偶关系



$$(A + B)(\bar{A} + C)(B + C)$$

$$= (A + B)(\bar{A} + C)$$

该公式说明：三个乘积项相加时，其中两个乘积项中，一项含有原变量A，另一项含有反变量 $\bar{A}$ ，而这两项的其余因子都是第三个乘积的因子，则第三个乘积项是多余的。



该公式可以推广为:

$$AB + \bar{A}C + BCDE = AB + \bar{A}C$$

(5) 关于异或和同或运算

对偶数个变量而言:

$$A_1 \oplus A_2 \oplus \dots \oplus A_n = \overline{A_1 \odot A_2 \odot \dots \odot A_n}$$

对奇数个变量而言:

$$A_1 \oplus A_2 \oplus \dots \oplus A_n = A_1 \odot A_2 \odot \dots \odot A_n$$



例证: $A_1 \oplus A_2 \oplus A_3 = A_1 \odot A_2 \odot A_3$

$$\begin{aligned} \text{证明: } A_1 \oplus A_2 \oplus A_3 &= \overline{A_1 \odot A_2} \oplus A_3 \\ &= \overline{A_1 \odot A_2} \cdot \overline{A_3} + \overline{\overline{A_1 \odot A_2}} \cdot A_3 \\ &= \overline{A_1 \odot A_2} \cdot \overline{A_3} + (A_1 \odot A_2) \cdot A_3 \\ &= A_1 \odot A_2 \odot A_3 \end{aligned}$$



异或和同或的其他性质：

$$A \oplus 0 = A$$

$$A \oplus 1 = \bar{A}$$

$$A \oplus A = 0$$

$$A \oplus (B \oplus C) = (A \oplus B) \oplus C$$

$$A(B \oplus C) = AB \oplus AC$$

$$A \odot 1 = A$$

$$A \odot 0 = \bar{A}$$

$$A \odot A = 1$$

$$A \odot (B \odot C) = (A \odot B) \odot C$$

$$A + (B \odot C) = (A + B) \odot (A + C)$$

对于所有公式，需要正确理解其含义，避免引起误解。

例如： $A(A+B)=A$ 不能得到 $A+B=1$

例如： $A \oplus (B+C) \neq A \oplus B + A \oplus C$



1.6 逻辑函数的标准形式

1.6.1 常用的逻辑函数式

$$F(A,B,C) = AB + \bar{A}C$$

与或式

$$= (A+C)(\bar{A}+B)$$

或与式

$$= \overline{\overline{AB}} \cdot \overline{\overline{\bar{A}C}}$$

与非-与非式

$$= \overline{\overline{A+C}} + \overline{\overline{\bar{A}+B}}$$

或非-或非式

$$= \overline{\overline{AC}} + \overline{\overline{AB}}$$

与或非式



1.6.2 函数的“与-或”式和“或-与”式

“与-或”式，指一个函数表达式中包含若干个“与”项，这些“与”项的“或”表示这个函数。

例：
$$F(A,B,C,D)=A+B\bar{C}+\bar{A}B\bar{C}D$$

“或-与”式，指一个函数表达式中包含若干个“或”项，这些“或”项的“与”表示这个函数。

例：
$$F(A,B,C)=(A+B)(\bar{A}+C)(\bar{A}+\bar{B}+\bar{C})$$



1.6.3 最小项和最大项

1. 最小项

(1) 最小项特点 最小项是特殊的“与”项。

① n 个变量构成的每个最小项，一定是包含 n 个因子的乘积项；

② 在各个最小项中，每个变量必须以原变量或反变量形式作为因子出现一次，而且仅出现一次。

例 有A、B两变量的最小项共有四项(2^2):

$$\bar{A}\bar{B} \quad \bar{A}B \quad A\bar{B} \quad AB$$



例 有A、B、C三变量的最小项共有八项(2^3):

$\overline{A}\overline{B}\overline{C}$ 、 $\overline{A}\overline{B}C$ 、 $\overline{A}B\overline{C}$ 、 $\overline{A}BC$ 、 $A\overline{B}\overline{C}$ 、 $A\overline{B}C$ 、 $AB\overline{C}$ 、 ABC

(2) 最小项编号

任一个最小项用 m_i 表示, m 表示最小项, 下标 i 为使该最小项为1的变量取值所对应的等效十进制数。

最小项	$\overline{A}\overline{B}\overline{C}$	$\overline{A}\overline{B}C$	$\overline{A}B\overline{C}$	$\overline{A}BC$	$A\overline{B}\overline{C}$	$A\overline{B}C$	$AB\overline{C}$	ABC
二进制数	000	001	010	011	100	101	110	111
十进制数	0	1	2	3	4	5	6	7
编号	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7

注意: 变量的数目、顺序



(3) 最小项的性质

三变量全部最小项真值表

A B C	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7	$F = \sum_{i=0}^{2^n-1} m_i$
	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	$\bar{A}B\bar{C}$	$\bar{A}BC$	$A\bar{B}\bar{C}$	$A\bar{B}C$	$AB\bar{C}$	ABC	
0 0 0	1	0	0	0	0	0	0	0	1
0 0 1	0	1	0	0	0	0	0	0	1
0 1 0	0	0	1	0	0	0	0	0	1
0 1 1	0	0	0	1	0	0	0	0	1
1 0 0	0	0	0	0	1	0	0	0	1
1 0 1	0	0	0	0	0	1	0	0	1
1 1 0	0	0	0	0	0	0	1	0	1
1 1 1	0	0	0	0	0	0	0	1	1

- ① 变量任取一组值，仅有一个最小项为1，其他最小项为零；
- ② n 变量的全体最小项之和为1；



③ 不同的最小项相与，结果为0；

④ 两最小项**相邻**，相邻最小项相“**或**”，可以合并成一项，并可以消去一个变量因子。

相邻的概念：两最小项如**仅有一个变量因子不同**，其他**变量均相同**，则称这两个最小项**相邻**。

相邻最小项相“或”的情况：

例： $\overline{A} \overline{B} \overline{C} + \overline{A} \overline{B} C = \overline{A} \overline{B}$

● 任一 **n** 变量的最小项，必定和其他 **n** 个不同最小项**相邻**。



2. 最大项

(1) 最大项特点 最大项是特殊的“或”项。

- ① **n**个变量构成的每个最大项，一定是包含**n**个因子的“或”项；
- ② 在各个最大项中，每个变量必须以原变量或反变量形式作为因子出现一次，而且仅出现一次。

例：有A、B两变量的最大项共有**四**项：

$$A + B \quad A + \bar{B} \quad \bar{A} + B \quad \bar{A} + \bar{B}$$



例：有A、B、C三变量的最大项共有八项：

$$\begin{aligned} &A+B+C, A+B+\bar{C}, A+\bar{B}+C, A+\bar{B}+\bar{C}, \\ &\bar{A}+B+C, \bar{A}+B+\bar{C}, \bar{A}+\bar{B}+C, \bar{A}+\bar{B}+\bar{C} \end{aligned}$$

(2) 最大项编号

任一个最大项用 M_i 表示， M 表示最大项，下标 i 为使该最大项为0的变量取值所对应的等效十进制数。

例：有最大项 $\bar{A} + B + C$ ，要使该最大项为0，A、B、C的取值应为1、0、0，二进制数100所等效的十进制数为4，所以 $\bar{A} + B + C = M_4$ 。



(3) 最大项的性质

三变量全部最大项真值表

A	B	C	M_0 $A+B+C$	M_1 $A+B+\bar{C}$	M_2 $A+\bar{B}+C$	M_3 $A+\bar{B}+\bar{C}$	M_4 $\bar{A}+B+C$	M_5 $\bar{A}+B+\bar{C}$	M_6 $\bar{A}+\bar{B}+C$	M_7 $\bar{A}+\bar{B}+\bar{C}$
0	0	0	0	1	1	1	1	1	1	1
0	0	1	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
1	1	0	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0

- ① 变量任取一组值，仅有一个最大项为0，其它最大项为1；
- ② n变量的全体最大项之积为0；



③ 不同的最大项相或，结果为 1；

④ 两**相邻**的最大项相“**与**”，可以合并成一项，并可以消去一个变量因子。

相邻的概念：两最大项如仅有一个变量因子不同，其他变量均相同，则称这两个最大项**相邻**。

相邻最大项相“与”的情况：

例： $(A+B+C)(A+B+\bar{C})=A+B$

● 任一 **n** 变量的最大项，必定和其他 **n** 个不同最大项**相邻**。



3. 最小项和最大项的关系

编号下标相同的最小项和最大项互为反函数，即

$$M_i = \overline{m_i} \quad \text{或} \quad m_i = \overline{M_i}$$

例如A、B、C三变量：

$$m_0 = \overline{A}\overline{B}\overline{C} = \overline{A+B+C} = \overline{M_0}$$

$$M_0 = A+B+C = \overline{\overline{A}\overline{B}\overline{C}} = \overline{m_0}$$



1.6.4 标准与或式和标准或与式

1. 逻辑函数的标准与或式

标准“与或”式又称为最小项之和形式，例：

$$\begin{aligned} F(A,B,C) &= \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC\bar{C} \\ &= \Sigma m(2, 4, 6) \\ &= \Sigma(2, 4, 6) \end{aligned}$$



- 任一逻辑函数都可以表达为最小项之和的形式，而且是唯一的。

例： $F(A,B,C) = A B + \bar{A} C$ 该式不是最小项之和形式

$$\begin{aligned} &= AB (\bar{C} + C) + \bar{A} C (\bar{B} + B) \\ &= AB\bar{C} + ABC + \bar{A}\bar{B}C + \bar{A}BC \\ &= m_6 + m_7 + m_1 + m_3 \\ &= \Sigma m (1, 3, 6, 7) \end{aligned}$$



2. 逻辑函数的标准或与式

标准“或与”式又称为最大项之积形式。

$$\begin{aligned}\text{例: } F(A,B,C) &= (A+B+C)(A+\bar{B}+C)(\bar{A}+B+C) \\ &= M_0 \cdot M_2 \cdot M_4 \\ &= \Pi M(0, 2, 4) = \Pi(0, 2, 4)\end{aligned}$$

● 任一逻辑函数都可以表达为最大项之积的形式，而且是唯一的。

$$\begin{aligned}\text{例: } F(A,B,C) &= (\bar{A} + C)(B + \bar{C}) \\ &= (\bar{A} + B \cdot \bar{B} + C)(A \cdot \bar{A} + B + \bar{C}) \\ &= (\bar{A} + B + C)(\bar{A} + \bar{B} + C)(A + B + \bar{C})(\bar{A} + B + \bar{C}) \\ &= M_4 \cdot M_6 \cdot M_1 \cdot M_5 = \Pi M(1, 4, 5, 6)\end{aligned}$$



3. 标准与或式和标准或与式的关系

$$\text{若 } F = \sum m_i \longrightarrow F(A,B,C) = m_1 + m_3 + m_4 + m_6 + m_7$$

$$\text{则 } \bar{F} = \sum_{j \neq i} m_j \longrightarrow \bar{F}(A,B,C) = m_0 + m_2 + m_5$$

$$F = \overline{\sum_{j \neq i} m_j} \longrightarrow F(A,B,C) = \overline{m_0 + m_2 + m_5}$$

$$= \prod_{j \neq i} \bar{m}_j \qquad = \bar{m}_0 \cdot \bar{m}_2 \cdot \bar{m}_5$$

$$= \prod_{j \neq i} M_j \qquad = M_0 \cdot M_2 \cdot M_5$$

■ 补充: $F'(A,B,C) = m_7 + m_5 + m_2 = \sum m(2, 5, 7)$

例: $F(A, B, C) = \sum m(1, 2, 4, 5) = \prod M(0, 3, 6, 7)$

例: $F(A, B, C) = \prod M(0, 2, 3, 7) = \sum m(1, 4, 5, 6)$



1.7 逻辑函数式与真值表

逻辑函数式与真值表都是表示逻辑函数的方法。

1.7.1 由逻辑函数式列真值表

由逻辑函数式列真值表可采用三种方法，以例说明：

例：试列出下列逻辑函数式的真值表。

$$F(A,B,C) = AB+BC$$

方法一：将A、B、C三变量的所有取值的组合（共八种），分别代入函数式 $F(A,B,C)=AB+BC$ ，逐一算出函数值，填入真值表。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



方法二： 先将函数式F表示为最小项之和的形式：

$$F(A,B,C) = AB+BC$$

$$= AB (C+\bar{C}) + (A+\bar{A}) BC$$

$$= ABC+AB\bar{C}+\bar{A}BC$$

$$= \Sigma m (3, 6, 7)$$

再根据最小项的性质，在真值表中对应于ABC取值为011、110、111处填“1”，其它位置填“0”。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



方法三： 根据函数式F的含义，直接填表。

函数 $F(A,B,C) = AB + BC$ 表示的含义为：

- ① 当**A**和**B**同时为“1”，即
 $AB=1$ 时， $F=1$ ；
- ② 当**B**和**C**同时为“1”，即
 $BC=1$ 时， $F=1$ ；
- ③ 当不满足上面两种情况时，
 $F=0$ 。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

方法三是一种较好的方法，要熟练掌握。



例： $F = \overline{(A \oplus B)(B \oplus C)}$

令： $F_1 = (A \oplus B)$; $F_2 = (B \oplus C)$

$$F = \overline{F_1 F_2}$$

A	B	C	F_1	F_2	F
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	1	1	0
0	1	1	1	0	1
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	0	1	1
1	1	1	0	0	1



1.7.2 由真值表写出逻辑函数式

根据最小项的性质，用观察法，可直接从真值表写出函数的最小项之和表达式。

例：已知函数F的真值表如下，求逻辑函数表达式。

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1



A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

解：由真值表可见，当ABC取**010**、**100**、**110**、**111**时，F为“**1**”。

所以，F由**4**个最小项组成：

$$F(A, B, C) = \sum m(2, 4, 6, 7)$$

$$=\overline{A}B\overline{C} + A\overline{B}\overline{C} + AB\overline{C} + ABC$$



1.8 逻辑函数的化简

化简的意义：

- ▲ 节省元器件, 降低电路成本;
- ▲ 提高电路可靠性;
- ▲ 减少连线, 制作方便。

最简**与或**表达式的标准:

- ① 所得**与或**表达式中, **乘积项** (与项) 数目最少;
- ② 每个乘积项中所含的**变量数**最少。

例:
$$F = A + AB + \bar{A}CD = A + \bar{A}CD = A + CD$$



1.8.1 公式化简法

针对某一逻辑式, 反复运用逻辑代数公式消去**多余的乘积项**和每个乘积项中**多余的因子**, 使函数式符合**最简标准**。

化简中常用方法:

1. 并项法

利用公式

$$AB + A\bar{B} = A$$

例: $F = A\bar{B}C + \bar{A}BC + ABC + \bar{A}\bar{B}C$

$$= (A\bar{B} + \bar{A}B)C + (AB + \bar{A}\bar{B})C$$

$$= (A \oplus B)C + (A \odot B)C$$

$$= (A \oplus B)C + \overline{(A \oplus B)}C$$

$$= C$$

在化简中注意代入规则的使用



2. 吸收法 利用公式 $A+AB=A$

例: $F = \bar{A} + \overline{\overline{A}BC} \overline{B+AC} + \bar{D} + BC$

$$= (\bar{A} + BC) + (\bar{A} + BC) \overline{B+AC} + \bar{D}$$

$$= \bar{A} + BC$$

反演律

3. 消项法 利用公式 $AB + \bar{A}C + BC = AB + \bar{A}C$

例: $F = \bar{A}\bar{B}C\bar{D} + \bar{A}E + BE + C\bar{D}E$

$$= \bar{A}\bar{B}C\bar{D} + (\bar{A} + B)E + C\bar{D}E$$

$$= \bar{A}\bar{B}C\bar{D} + \bar{A}\bar{B}E + C\bar{D}E$$

$$= \bar{A}\bar{B}C\bar{D} + (\bar{A} + B)E$$

$$= \bar{A}\bar{B}C\bar{D} + \bar{A}E + BE$$



4. 消因子法 利用公式 $A + \bar{A}B = A + B$

例：

$$\begin{aligned} F &= AB + \bar{A}C + \bar{B}C \\ &= AB + (\bar{A} + \bar{B})C \\ &= AB + \bar{A}\bar{B}C \\ &= AB + C \end{aligned}$$

5. 配项法 利用公式 $\bar{A} + A = 1$; $A \cdot 1 = A$ 等

例：

$$\begin{aligned} F &= AB + \bar{A}C + BC \\ &= AB + \bar{A}C + (A + \bar{A})BC \\ &= AB + \bar{A}C + ABC + \bar{A}BC \\ &= (AB + ABC) + (\bar{A}C + \bar{A}BC) \\ &= AB + \bar{A}C \end{aligned}$$



- 对比较复杂的函数式，要求熟练掌握上述方法，才能把函数化成最简。

1.8.2 卡诺图化简法

该方法是将逻辑函数用一种称为“**卡诺图**”的图形来表示，然后在卡诺图上进行函数的化简的方法。

1. **卡诺图**的构成



卡诺图是一种包含一些小方块的几何图形，图中每个小方块称为一个单元，每个单元对应一个最小项。最小项在卡诺图中的位置不是任意的，它必须满足相邻性规则。卡诺图中的相邻有两层含义：

- (1) 几何相邻性，即几何位置上相邻，也就是左右紧挨着或者上下相接；
- (2) 对称相邻性，即图形中对称位置的单元是相邻的。



▲ 卡诺图

二变量图

A	B	m_i
0	0	m_0
0	1	m_1
1	0	m_2
1	1	m_3



	$\bar{B}$	B
$\bar{A}$	$\bar{A}\bar{B}$	$\bar{A}B$
A	$A\bar{B}$	AB

	B	
	0	1
A		
0	m_0	m_1
1	m_2	m_3

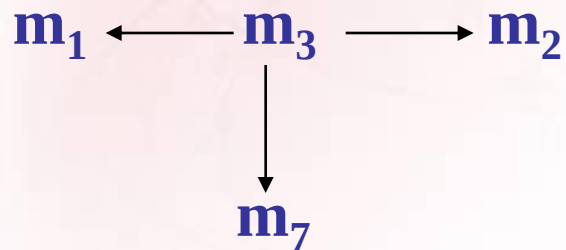


三变量图

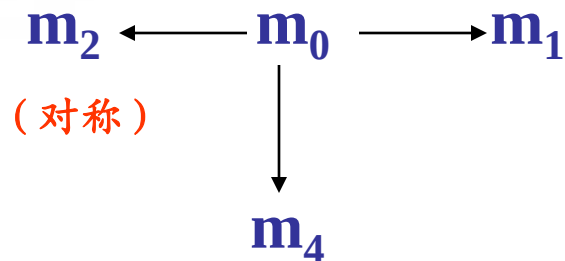
A \ BC	BC			
	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$ m_0	$\bar{A}\bar{B}C$ m_1	$\bar{A}BC$ m_3	$\bar{A}B\bar{C}$ m_2
1	$A\bar{B}\bar{C}$ m_4	$A\bar{B}C$ m_5	ABC m_7	$AB\bar{C}$ m_6

循环码

相邻性规则



相邻性规则

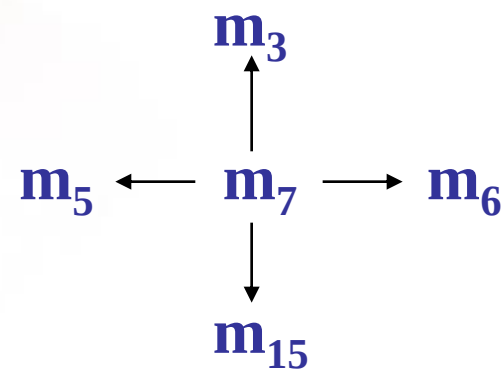




四变量图

AB \ CD	CD			
	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

相邻性规则





五变量图

AB \ CDE								
	000	001	011	010	110	111	101	100
00	0	1	3	2	6	7	5	4
01	8	9	11	10	14	15	13	12
11	24	25	27	26	30	31	29	28
10	16	17	19	18	22	23	21	20



2. 逻辑函数的卡诺图表示法

用卡诺图表示逻辑函数，只是把各组变量值所对应的逻辑函数F的值，填在对应的小方格中。

（其实卡诺图是真值表的另一种画法）

例： $F(A, B, C) = \bar{A}BC + A\bar{B}C + ABC$

用卡诺图表示为：

A \ BC	BC			
	00	01	11	10
0	0	0	1 m_3	0
1	0	1 m_5	1 m_7	0



例：画出 $F(A,B,C,D)=\overline{A}\overline{B}\overline{C}\overline{D}+B\overline{C}D+\overline{A}\overline{C}+A$
用卡诺图表示为：

AB \ CD		CD			
		00	01	11	10
00	00	1	1	0	0
	01	1	1	0	0
11	11	1	1	1	1
	10	1	1	1	1



3. 在卡诺图上合并最小项的规则

当卡诺图中有最小项相邻时（即：有标1的方格相邻），可利用最小项相邻的性质，对最小项合并。

规则为：

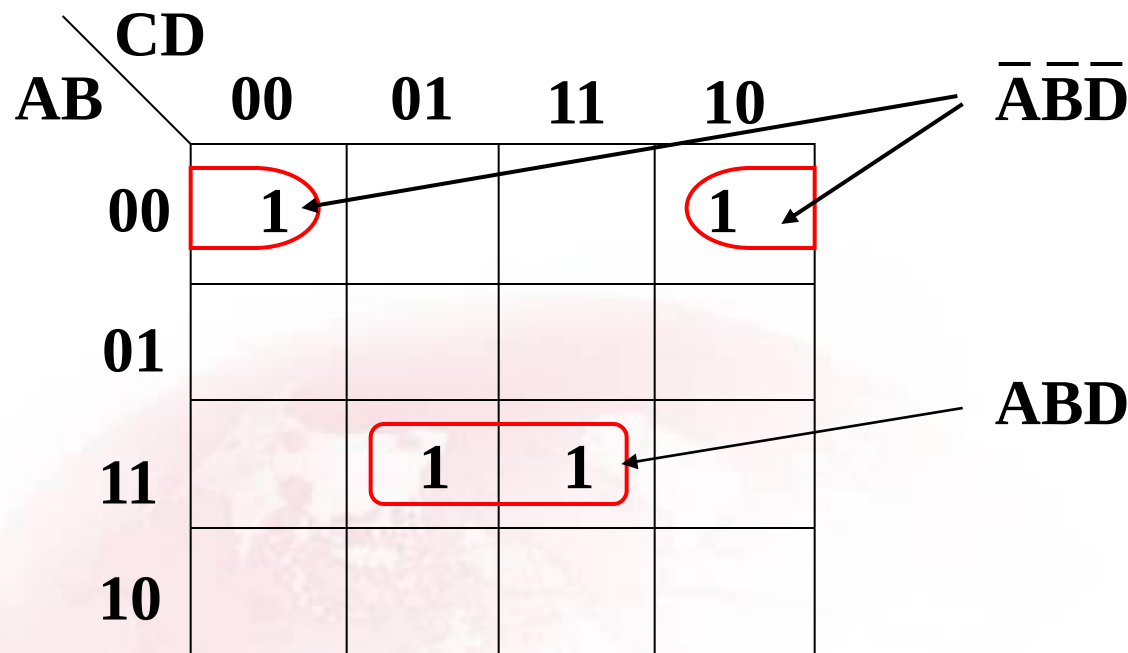
(1) 卡诺图上任何两个标1的方格相邻，可以合为1项，并可消去1个变量。

例：

A \ BC				
	00	01	11	10
0	0	0	1	0
1	0	1	1	0

$$\bar{A}BC + ABC = BC$$

$$A\bar{B}C + ABC = AC$$



(2) 卡诺图上任何四个标1方格相邻，可合并为一项，并可消去两个变量。

四个标1方格相邻的特点：

① 同在一行或一列

② 同在一田字格中



例:

AB \ CD	$\bar{A}\bar{B}$			
	00	01	11	10
00	1	1	1	1
01			1	
11			1	
10			1	

Diagram illustrating a Karnaugh map for a 4-variable function. The map shows four groups of 1s:

- A group of four 1s in the top row (AB=00), labeled $\bar{A}\bar{B}$.
- A group of four 1s in the third column (CD=11), labeled CD .

同在一行或一列

AB \ CD	$\bar{B}\bar{D}$			
	00	01	11	10
00	1			1
01		1	1	
11		1	1	
10	1			1

Diagram illustrating a Karnaugh map for a 4-variable function. The map shows four groups of 1s:

- A group of four 1s in the corners (AB=00, 10 and CD=00, 10), labeled $\bar{B}\bar{D}$.
- A group of four 1s in the middle (AB=01, 11 and CD=01, 11), labeled BD .

同在一个田字格中



思考题

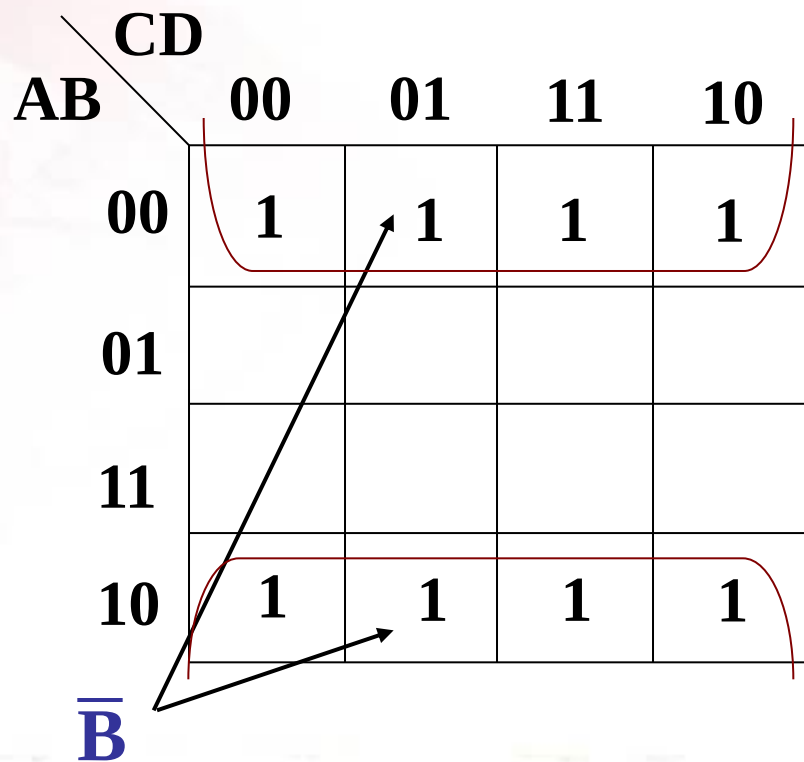
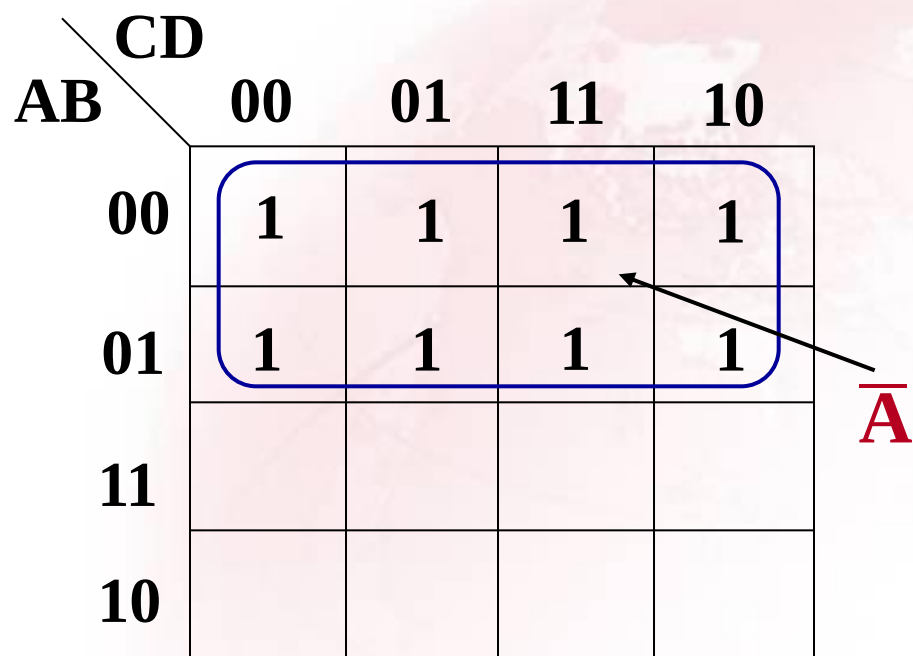
AB \ CD	CD			
	00	01	11	10
00	1	1	1	1
01			1	1
11			1	1
10				

AB \ CD	CD			
	00	01	11	10
00			1	1
01	1			1
11	1			1
10			1	1



(3) 卡诺图上任何八个标1的方格相邻，可以并为一项，并可消去三个变量。

例：





思考题：

AB \ CD	CD			
	00	01	11	10
00		1	1	
01		1	1	
11		1	1	
10		1	1	

综上所述，在 n 个变量的卡诺图中，只有**2的 i 次方个相邻的标1方格**（必须排列成方形格或矩形格的形状）才能圈在一起，合并为一项，该项**保留了原来各项中 $n-i$ 个相同的变量，消去 i 个不同变量**。



4. 用卡诺图化简逻辑函数（化为最简与或式）

（1）最简标准：

- ▲ 与项数最少，意味着卡诺图中圈数最少；
- ▲ 每项中的变量数最少，意味着卡诺图中的圈尽可能大。



例：将 $F(A, B, C) = \Sigma m(3, 4, 5, 6, 7)$
化为最简与或式。

BC \ A	00	01	11	10
0			1	
1	1	1	1	1

$$F = A + BC \quad (\text{最简})$$

BC \ A	00	01	11	10
0			1	
1	1	1	1	1

$$F = A\bar{B} + BC + AB\bar{C} \quad (\text{非最简})$$

$$\{ = A(\bar{B} + B\bar{C}) + BC$$

$$= A(\bar{B} + \bar{C}) + BC$$

$$= A\bar{B}\bar{C} + BC$$

$$= A + BC \}$$



(2) 化简步骤（结合举例说明）

例：将 $F(A,B,C,D) = \sum m(0,1,3,7,8,10,13)$ 化为最简与或式。

解：① 由表达式填卡诺图；

② 圈出孤立的标1方格；

m_{13}

AB \ CD		CD			
		00	01	11	10
AB	00	1	1	1	
	01			1	
	11		1		
	10	1			1



CD \ AB		CD				
		00	01	11	10	
00	1	1	1			$\bar{A}\bar{B}\bar{C}$ $\bar{A}CD$
01				1		
11			1			$AB\bar{C}D$
10	1				1	$A\bar{B}\bar{D}$

- ③ 找出只被一个最大的圈所覆盖的标1方格，并圈出覆盖该标1方格的最大圈； **m_7, m_{10}**
- ④ 将剩余的相邻标1方格，圈成尽可能少，而且尽可能大的圈。 **m_0, m_1**



⑤ 将各个对应的乘积项相加, 写出最简与或式。

$$F(A,B,C,D)=ABC\bar{C}D+\bar{A}CD+A\bar{B}\bar{D}+\bar{A}\bar{B}\bar{C}$$

例: $F(A,B,C,D)=\bar{A}\bar{C}+\bar{A}C\bar{D}+ABD+\bar{B}\bar{C}+\bar{B}C\bar{D}$

		CD			
		00	01	11	10
AB	00	1	1		1
	01	1	1		1
	11		1	1	
	10	1	1		1

$$F(A,B,C,D)=ABD+\bar{B}\bar{D}+\bar{A}\bar{D}+\bar{C}\bar{D}$$

可以得到最简与非-与非式:

$$F(A,B,C,D)=\overline{\overline{ABD} \cdot \overline{\bar{B}\bar{D}} \cdot \overline{\bar{A}\bar{D}} \cdot \overline{\bar{C}\bar{D}}}$$



一种特殊情况：

AB \ C	C	
	0	1
00		1
01	1	1
11	1	
10	1	1

$$F = A\bar{B} + B\bar{C} + \bar{A}C$$

AB \ C	C	
	0	1
00		1
01	1	1
11	1	
10	1	1

$$F = \bar{A}B + \bar{B}C + AC$$

得到两种化简结果，也都是最简的。



(3) 化简中注意的问题

- ① 每一个标1的方格必须至少被圈一次；
- ② 每个圈中包含的相邻小方格数，必须为2的整数次幂；
- ③ 为了得到尽可能大的圈，圈与圈之间可以部分重叠；

		CD			
		00	01	11	10
AB	00	1	1		1
	01	1	1		1
	11		1	1	
	10	1	1		1



④ 若某个圈中的所有标1方格，已经完全被其它圈所覆盖，则该圈为多余的。

例如：

$$F = \bar{A}\bar{B}\bar{C} + \bar{A}CD + A\bar{C}D + ABC$$

最简

$$F = \bar{A}\bar{B}\bar{C} + \bar{A}CD + A\bar{C}D + ABC + (BD)$$

蓝色的圈为多余的。

AB \ CD		CD			
		00	01	11	10
AB	00			1	
	01	1	1	1	
	11		1	1	1
	10		1		



(4) 用卡诺图求反函数的最简与或式

方法：在卡诺图中合并标 0 方格，可得到反函数的最简与或式。

例：用卡诺图求 $F(A,B,C)=AB+BC+CA$ 的反函数 $\bar{F}$ 的最简与或式

BC		00	01	11	10
A	0	0	0	1	0
	1	0	1	1	1

$$\bar{F} = \bar{A}\bar{B} + \bar{B}\bar{C} + \bar{A}\bar{C}$$



- 常利用该方法来求逻辑函数F的最简与或非式，例如将上式F上的非号移到右边，就得到 F 的最简与或非表达式。

$$F = \overline{\overline{A}\overline{B} + \overline{B}\overline{C} + \overline{A}\overline{C}}$$

(5) 用卡诺图对复杂逻辑函数进行化简的技巧

对较为复杂的逻辑函数，可将函数分解成多个部分，将每个部分分别填入各自的卡诺图中，然后通过卡诺图对应方格的运算，求出函数的卡诺图，再进行化简。



例：化简逻辑函数

$$F(A,B,C,D)=(AB+\bar{A}C+\bar{B}D)\oplus(A\bar{B}\bar{C}D+\bar{A}CD+BCD+\bar{B}C)$$

CD		CD				$\oplus$	CD		$=$	CD									
		00	01	11	10														
AB	00		1	1	1	AB <th>00</th> <td></td> <td></td> <td>1</td> <td>1</td> <td rowspan="4">AB<th>00</th><td></td><td>1</td><td></td><td></td></td>	00			1	1	AB <th>00</th> <td></td> <td>1</td> <td></td> <td></td>	00		1				
	01			1	1		01 <td></td> <td></td> <td>1</td> <td></td> <td rowspan="4">01<td></td><td></td><td></td><td>1</td></td>			1			01 <td></td> <td></td> <td></td> <td>1</td>				1		
	11	1	1	1	1			11 <td></td> <td></td> <td>1</td> <td></td> <td rowspan="4">11<td>1</td><td>1</td><td></td><td>1</td></td>			1				11 <td>1</td> <td>1</td> <td></td> <td>1</td>	1	1		1
	10		1	1					10 <td></td> <td>1</td> <td>1</td> <td>1</td> <td rowspan="4">10<td></td><td></td><td></td><td>1</td></td>		1			1		1	10 <td></td> <td></td> <td></td> <td>1</td>		

$$F(A,B,C,D)=\bar{A}\bar{B}\bar{C}D+AB\bar{C}+BC\bar{D}+AC\bar{D}$$



1.8.3 不完全确定的逻辑函数及其化简

在某些实际数字电路中，逻辑函数的输出只和一部分最小项有确定对应关系，而和余下的最小项无关。余下的最小项无论写入逻辑函数式还是不写入逻辑函数式，都不影响电路的逻辑功能。把这些最小项称为**无关项**，用英文字母**d**(don't care)表示，对应的函数值记为“**×**”。

包含**无关项**的逻辑函数称为**不完全确定**的逻辑函数。



利用**不完全确定**的逻辑函数中的**无关项**往往可以将函数化得更简单。

例：设计一个奇偶判别电路。电路输入为8421BCD码，当输入为偶数时，输出为0；当电路输入为奇数时，输出为1。

由于8421BCD码中无**1010~1111**这6个码，电路禁止输入这6个码，这6个码对应的最小项为**无关项**。



真值表



A	B	C	D	F	A	B	C	D	F
0	0	0	0	0	1	0	0	0	0
0	0	0	1	1	1	0	0	1	1
0	0	1	0	0	1	0	1	0	×
0	0	1	1	1	1	0	1	1	×
0	1	0	0	0	1	1	0	0	×
0	1	0	1	1	1	1	0	1	×
0	1	1	0	0	1	1	1	0	×
0	1	1	1	1	1	1	1	1	×

$$F(A,B,C,D)=\Sigma m(1,3,5,7,9)+\Sigma d(10 \sim 15)$$



$$F(A,B,C,D)=\Sigma m(1,3,5,7,9)+\Sigma d(10 \sim 15)$$

若不利用无关项（即将卡诺图中的 $\times$ 均作0处理），则化简结果为：

$$F(A,B,C,D)=\bar{A}D+\bar{B}\bar{C}D$$

若利用无关项（即将卡诺图中的 $\times$ 按化简的需要任意处理，将有些 $\times$ 当作0，有些 $\times$ 当作1），则化简结果为：

$$F(A,B,C,D)=D$$

AB \ CD	CD			
	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	$\times$	$\times$	$\times$	$\times$
10	0	1	$\times$	$\times$

完整地将函数写为：

$$\begin{cases} F(A,B,C,D)=D \\ \Sigma d(10 \sim 15)=0 \end{cases}$$



例: $F=(A \oplus B)C\bar{D}+\bar{A}B\bar{C}+\bar{A}\bar{C}D$, 且 $AB+CD=0$

AB \ CD	00	01	11	10
00	0	1	×	0
01	1	1	×	1
11	×	×	×	×
10	0	0	×	1

$$F(A,B,C,D)=B+\bar{A}D+AC$$

注意: 在无特殊说明的情况下, 为使逻辑函数化的更简单, 均应按上述**第二种**方法处理最小项。



1.8.4 逻辑函数式化简为其它形式

以 $F(A,B,C) = AB + \bar{A}C$ 为例，说明如何将与或式转换为其他类型的表达式。

$$F(A,B,C) = AB + \bar{A}C$$

与或式

$$= (\bar{A} + B)(A + C)$$

或与式

$$= \overline{\overline{AB}} \cdot \overline{\overline{\bar{A}C}}$$

与非 - 与非式

$$= \overline{\overline{\bar{A} + B} + \overline{\overline{A + C}}}$$

或非 - 或非式

$$= \overline{\overline{AB} + \overline{\bar{A}C}}$$

与或非式



1. 与非—与非式

由最简的与或式，经过两次求反，可得与非—与非式

$$\begin{aligned} F(A,B,C) &= AB + \bar{A}C && \text{(与或式)} \\ &= \overline{\overline{AB + \bar{A}C}} \\ &= \overline{\overline{AB}} \cdot \overline{\overline{\bar{A}C}} && \text{(与非—与非式)} \end{aligned}$$



2. 与或非式

求出 $F = AB + \bar{A}C$ 反函数 $\bar{F}$ 的最简与或式，再对 $\bar{F}$ 求反

BC					
		00	01	11	10
A	0	0	1	1	0
	1	0	0	1	1

$$\bar{F} = A\bar{B} + \bar{A}\bar{C}$$

$$F = \overline{(\bar{F})} = \overline{A\bar{B} + \bar{A}\bar{C}} \quad (\text{与或非式})$$

$$= \overline{\overline{A}\overline{B}} \overline{\overline{A}\overline{C}}$$

$$= (\overline{A} + B)(A + C) \quad (\text{或与式})$$

$$= \overline{\overline{(\overline{A} + B)(A + C)}}$$

$$= \overline{\overline{A} + B} \overline{A + C} \quad (\text{或非-或非式})$$



3. 或与式

由最简的与或式，运用两次求对偶或两次求反可得或与式。

例：求 $F = AB + \bar{A}C$ 的最简或与式。

(1) 两次求对偶

$$F' = (A + B)(\bar{A} + C) = \bar{A}B + AC$$

$$F = (F')' = (\bar{A} + B)(A + C)$$

(2) 两次求反

$$\bar{F} = A\bar{B} + \bar{A}\bar{C}$$

$$F = \overline{(\bar{F})} = \overline{A\bar{B} + \bar{A}\bar{C}} = (\bar{A} + B)(A + C)$$



4. 或非—或非式

由最简的或与式，经过两次求反，可得或非—或非式

$$\begin{aligned} F &= (\bar{A} + B)(A + C) \\ &= \overline{\overline{(\bar{A} + B)(A + C)}} \\ &= \overline{\bar{A} + B} \overline{A + C} \end{aligned}$$



※ 1.8.5 奎恩—麦克拉斯基化简法（Q—M法）

Q-M法有确定的流程，适用于任何复杂逻辑函数的化简。

1. 将函数化为最小项之和的形式，列出最小项编码表；
2. 按包含1的个数将最小项分组；
3. 合并相邻的最小项；
4. 选择最少的乘积项；
5. 最后确定化简结果中的乘积项。



1.8.6 多输出逻辑函数的化简

实际的数字电路，常常是一个多输出电路，即对应于相同一组输入变量，存在多个输出函数。

多输出函数的化简也是以单个函数的化简方法为基础，但要考虑到整体电路最简。



例:
$$\begin{cases} F_1(A,B,C) = \sum m(1,4,5) \\ F_2(A,B,C) = \sum m(1,3,7) \end{cases}$$

若按单个函数化简方法

BC \ A	00	01	11	10
0		1		
1	1	1		

BC \ A	00	01	11	10
0		1	1	
1			1	

化简的结果为:
$$\begin{cases} F_1 = A\bar{B} + \bar{B}C \\ F_2 = \bar{A}C + BC \end{cases}$$



从整体出发，考虑函数的化简

BC \ A	00	01	11	10
0		1		
1	1	1		

BC \ A	00	01	11	10
0		1	1	
1			1	

化简的结果为：

$$\begin{cases} F_1 = \bar{A}\bar{B}C + A\bar{B} \\ F_2 = \bar{A}\bar{B}C + BC \end{cases}$$



习 题

1. $(01111000.0101)_{8421BCD} = (\textcolor{red}{1001110.1})_2$;

2. $(1000)_{5421BCD} + (1101)_{2421BCD} = (\textcolor{red}{00010010})_{8421BCD}$

3. 逻辑函数 $F(A,B,C) = \sum m(0,3,5)$, 则其反函数 $\bar{F}(A,B,C) = \sum m(\textcolor{red}{1,2,4,6,7})$, 对偶函数 $F'(A,B,C) = \sum m(\textcolor{red}{0,1,3,5,6})$;

4. 欲将二进制码 $B_3B_2B_1B_0$ 转换为四位格雷码 $R_3R_2R_1R_0$, 则 $R_3 = (\textcolor{red}{B_3})$, $R_0 = (\textcolor{red}{B_1 \oplus B_0})$;

5. $F(A,B,C,D) = \overline{(AB+CD)(BC+AD)} + \bar{A}BC$ 的最简与或表达式为 $F(A,B,C,D) = (\textcolor{red}{\bar{A} + \bar{B}\bar{C} + \bar{C}\bar{D} + \bar{B}\bar{D}})$, 其反函数的最小项之和表达式为 $\bar{F}(A,B,C,D) = \sum m(\textcolor{red}{11,13,14,15})$ 。



第1章 小结

- 常用数制、编码以及之间的相互转换
- 逻辑运算以及实现逻辑运算的门电路
- 基本定律和规则
- 逻辑函数的标准形式
 - 最小项之和的形式、最大项之积的形式
- 真值表、逻辑函数式及其相互表示
- 逻辑函数的化简